

Gradients, Edges & Lines

CS 4243 Computer Vision & Pattern Recognition

Angela Yao

Recap & Outline

Last Week

- Point processing
- Histogram operations
- Linear filtering, cross-correlation vs. convolution operations
- Denoising, sharpening, template matching
- Non-linear filtering, e.g. median, max filters

Today's Lecture

- Motivation for studying image edges
- Derivative filters to extract gradients
- Need for smoothing
- Canny edge detector
- Line detection with Hough transform

Why Study Edges?

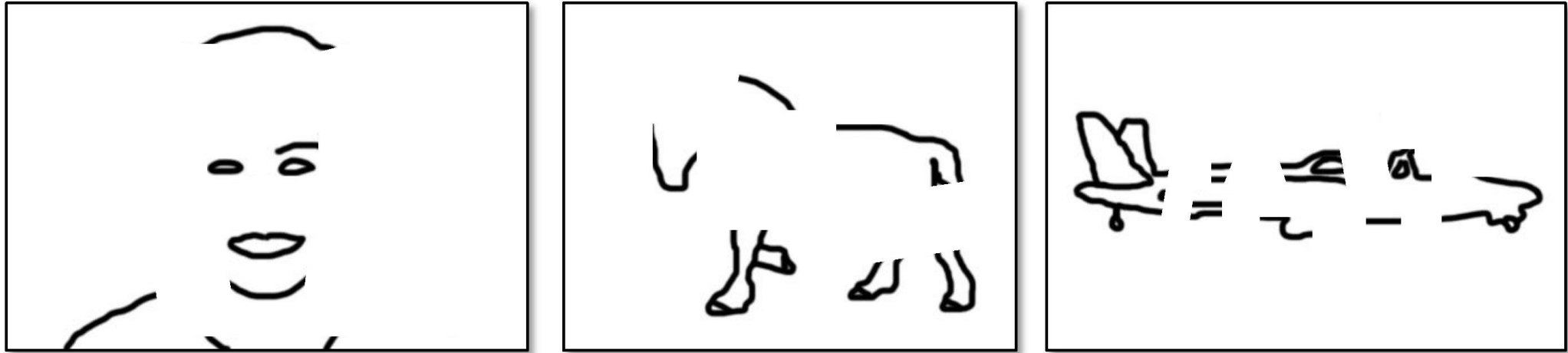


Figure from J. Shotton et al., PAMI 2007

- Human visual system is sensitive to edges
- Edges compactly convey salient information; cues for recognition
- Edges are resilient to changes in lighting and colour

Image Edges Convey Information

reflectance changes,
textures & appearances

depth discontinuities &
object boundaries

change in surface
orientations & shape

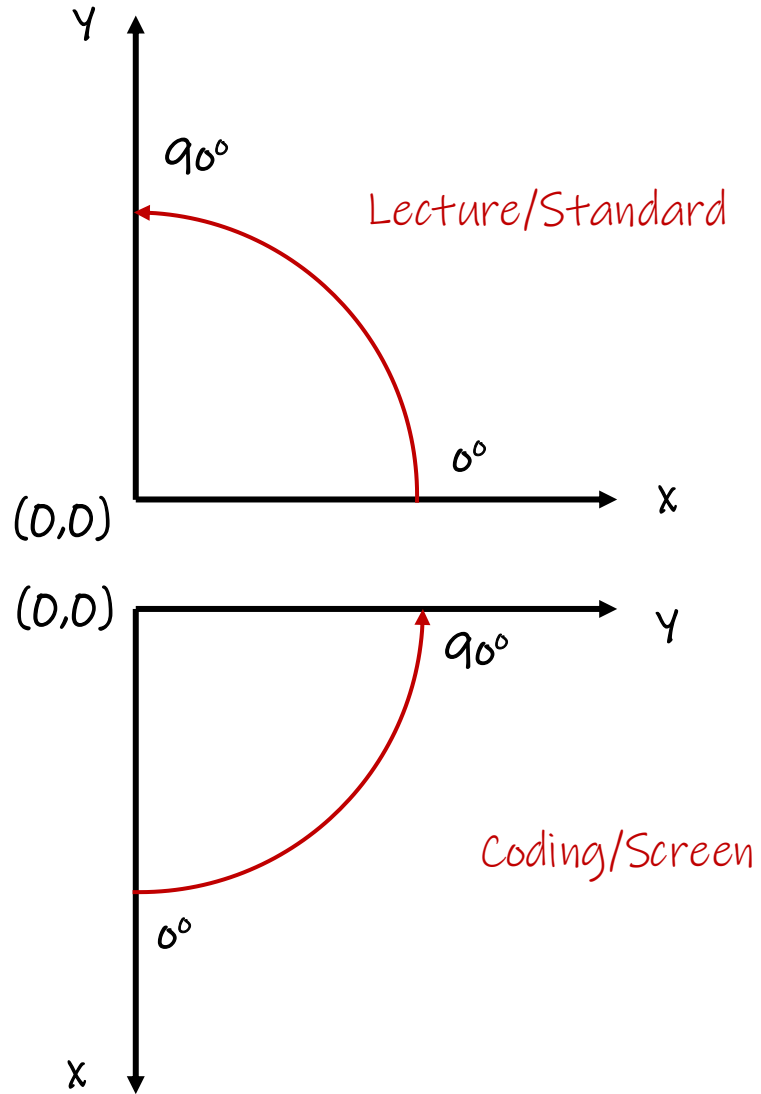


shadows

Image Gradients

Definition, magnitude & orientation, Sobel filtering

Axes/Orientation Conventions

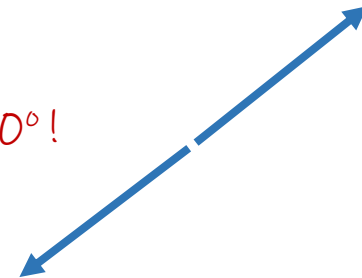


Where do the gradients point?



- To define a direction properly we need the full 0 to 360° range.
- What about lines? How can we uniquely identify a line or edge?

Only need 0 to 180°!



What are Image Edges or Gradients?

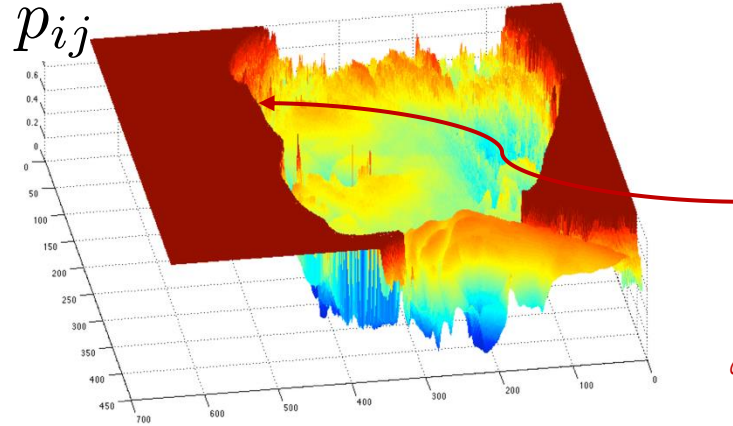
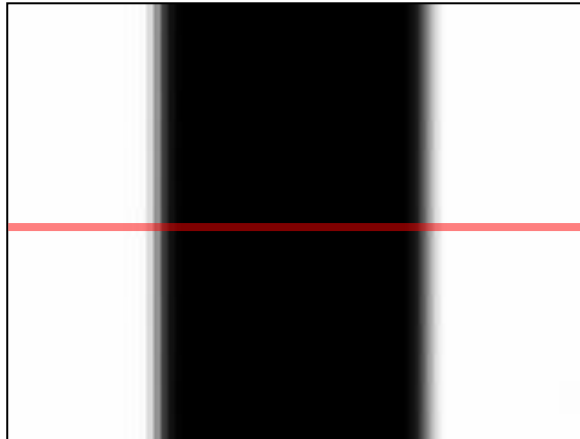


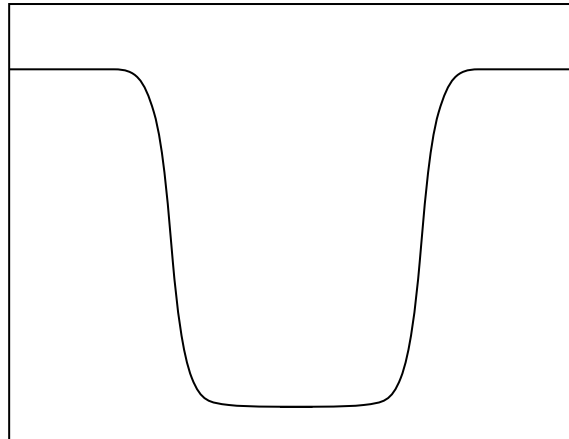
Image edges are sharp discontinuities in the intensity.

Derivatives are large at discontinuities. Edges correspond to extrema of derivative.

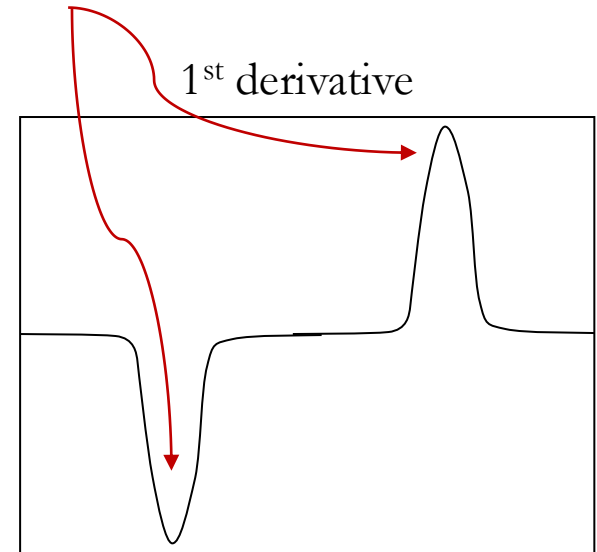
image



intensity along horizontal scanline

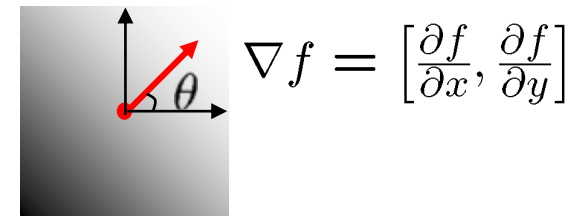
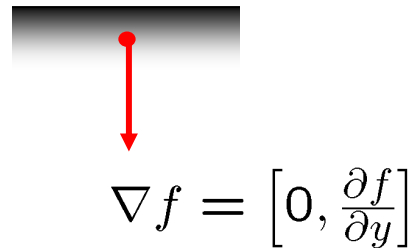
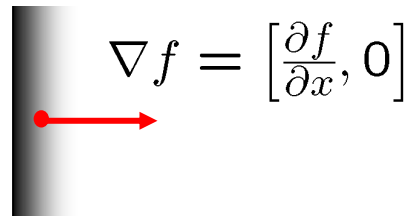


1st derivative



Understanding Image Gradients

$\nabla f = \left[\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y} \right]$ The gradient is a vector that points in the direction of most rapid change in intensity.



The **gradient direction** (orientation of edge normal) is given by:

$$\theta = \tan^{-1} \left(\frac{\partial f}{\partial y} / \frac{\partial f}{\partial x} \right)$$

The **edge strength** is given by the gradient magnitude

$$\|\nabla f\| = \sqrt{\left(\frac{\partial f}{\partial x} \right)^2 + \left(\frac{\partial f}{\partial y} \right)^2}$$

Images are discrete. How to take (partial) derivatives of discrete signals?

Take finite differences.

Finite differences

Recall: definition of a derivative (with a forward difference)

$$f'(x) = \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h}$$

Alternative with central difference

$$f'(x) = \lim_{h \rightarrow 0} \frac{f(x+0.5h) - f(x-0.5h)}{h}$$

Discrete version: remove limit, set $h = 2$

$$f'(x) = \frac{f(x+1) - f(x-1)}{2}$$

What filter kernel does this correspond to?

1D derivative filter

correlation	<table><tr><td>-1</td><td>0</td><td>1</td></tr></table>	-1	0	1
-1	0	1		
convolution	<table><tr><td>1</td><td>0</td><td>-1</td></tr></table>	1	0	-1
1	0	-1		

The Sobel filter

Horizontal Sobel filter:

Horizontal gradients $\rightarrow$ vertical lines

1	0	-1
2	0	-2
1	0	-1

=

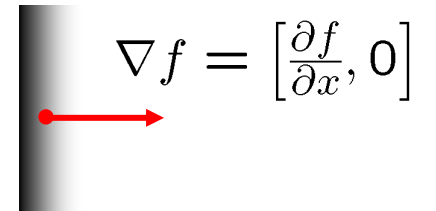
1
2
1

*

1	0	-1
---	---	----

1D derivative
filter

What filter is this? *blurring*



Vertical Sobel filter:

Vertical gradients $\rightarrow$ horizontal lines

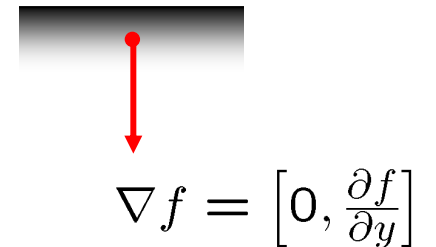
1	2	1
0	0	0
-1	-2	-1

=

1	2	1
---	---	---

*

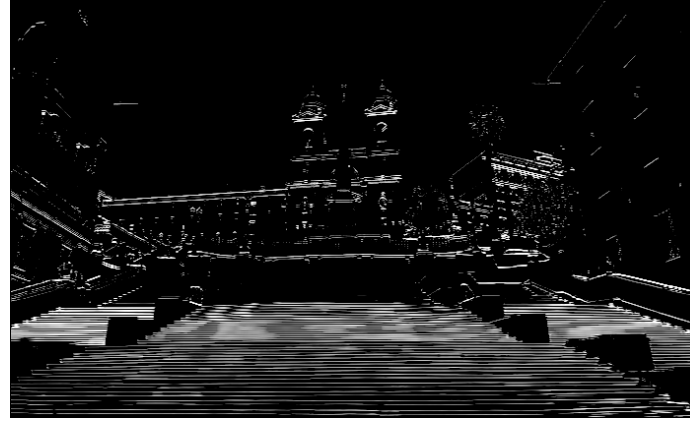
1
0
-1



Sobel filter example



Original Image



Vertical Sobel filter



Horizontal Sobel filter

Computing Image Gradients

1. Select your favorite gradient filters.

$$\mathbf{S}_x = \begin{bmatrix} 1 & 0 & -1 \\ 2 & 0 & -2 \\ 1 & 0 & -1 \end{bmatrix}$$

$$\mathbf{S}_y = \begin{bmatrix} 1 & 2 & 1 \\ 0 & 0 & 0 \\ -1 & -2 & -1 \end{bmatrix}$$

2. Convolve with the image $\mathbf{f}$ to compute image gradient components $\mathbf{G}_x$, $\mathbf{G}_y$.

$$\mathbf{G}_x = \frac{\partial \mathbf{f}}{\partial x} = \mathbf{S}_x * \mathbf{f} \quad \mathbf{G}_y = \frac{\partial \mathbf{f}}{\partial y} = \mathbf{S}_y * \mathbf{f}$$

3. Compute gradient orientation and magnitude.

$$\begin{array}{ccc} \nabla \mathbf{f} = \left[\frac{\partial \mathbf{f}}{\partial x}, \frac{\partial \mathbf{f}}{\partial y} \right] & \theta = \tan^{-1} \left(\frac{\partial \mathbf{f}}{\partial y} / \frac{\partial \mathbf{f}}{\partial x} \right) & \mathbf{M} = \|\nabla \mathbf{f}\| = \sqrt{\left(\frac{\partial \mathbf{f}}{\partial x} \right)^2 + \left(\frac{\partial \mathbf{f}}{\partial y} \right)^2} \\ \text{gradient} & \text{orientation} & \text{magnitude} \end{array}$$



Image Gradients & Orientation



Original Image



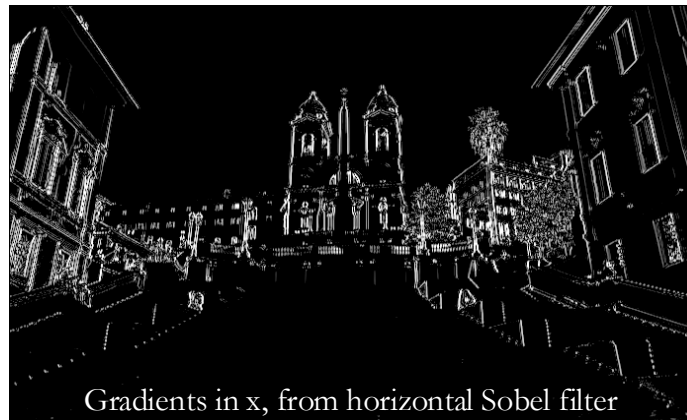
Gradients in y from vertical Sobel filter

$$\mathbf{G}_y = \frac{\partial f}{\partial y} = S_y * f$$



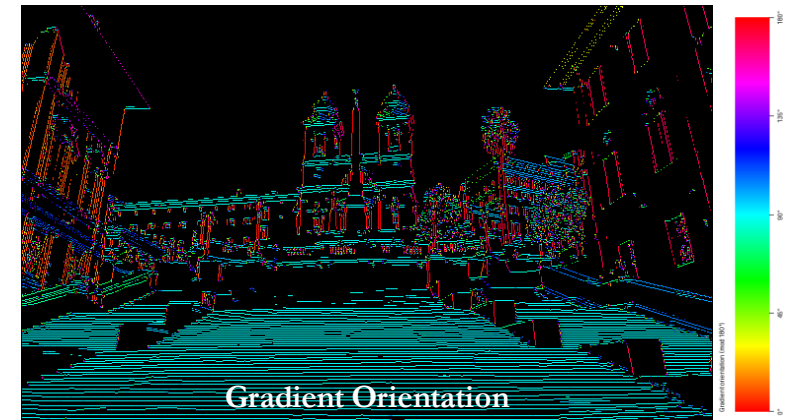
Gradient Magnitude

$$M = \|\nabla f\| = \sqrt{\left(\frac{\partial f}{\partial x}\right)^2 + \left(\frac{\partial f}{\partial y}\right)^2}$$



Gradients in x, from horizontal Sobel filter

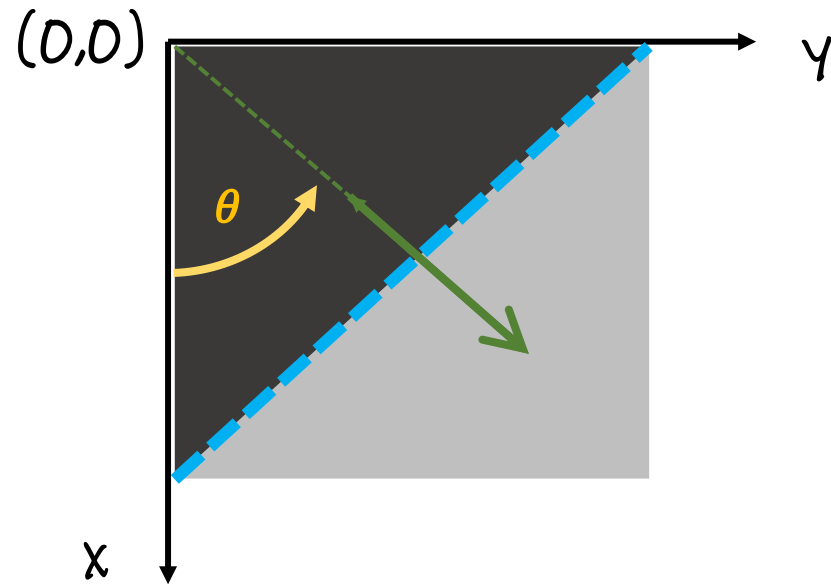
$$\mathbf{G}_x = \frac{\partial f}{\partial x} = S_x * f$$



Gradient Orientation

$$\theta = \tan^{-1} \left(\frac{\partial f / \partial y}{\partial f / \partial x} \right)$$

Gradient Orientation



The gradient points across the edge, i.e. the direction is normal to the edge.

The gradient points from darker intensities toward brighter intensities.

- Where is the edge?
- What direction is the gradient orientation pointing towards?
- Gradient orientation is given mathematically as

$$\theta = \tan^{-1} \left(\frac{\frac{\partial f}{\partial y}}{\frac{\partial f}{\partial x}} \right)$$

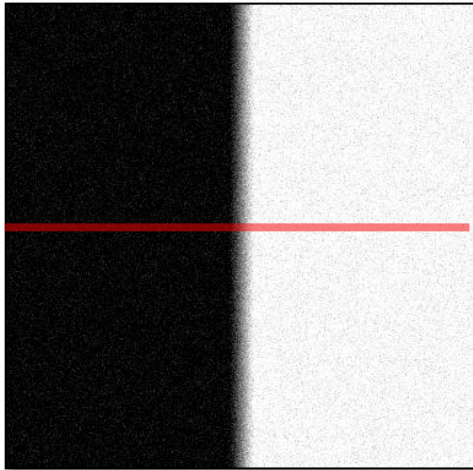
Compute as $\text{atan2}(G_y, G_x)$ instead of the division; this preserves the quadrant and is defined when G_x is 0.

- Gradient orientation is not the same as edge orientation!

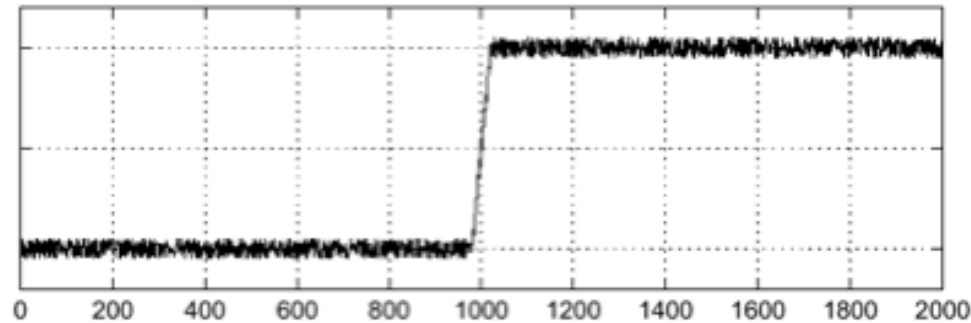
Gaussian Derivative & Laplace Filters

Derivative of Gaussians, Laplacian of Gaussians

How do you find the edge in this signal?

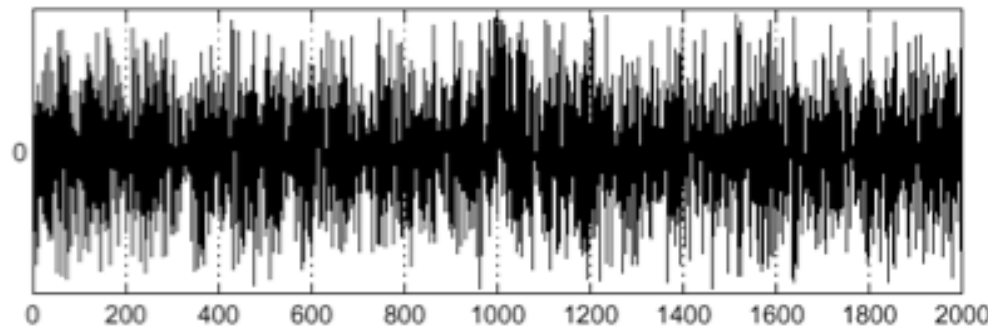


intensity plot



Using a derivative filter:

Where is the
extrema here?



Noise is high-frequency.
Differentiation accentuates noise

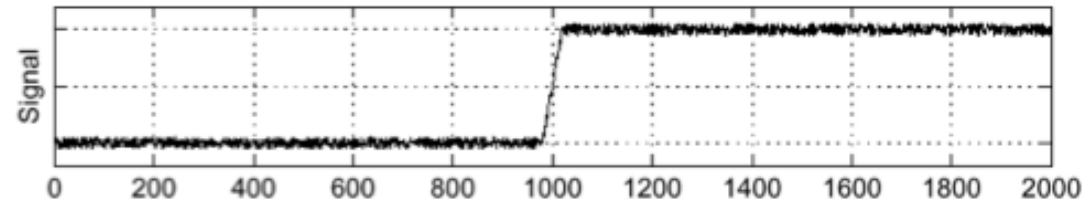
$$\frac{d \sin \omega x}{dx} = \omega \cos \omega x$$

Multiplicative factor
proportional to frequency!

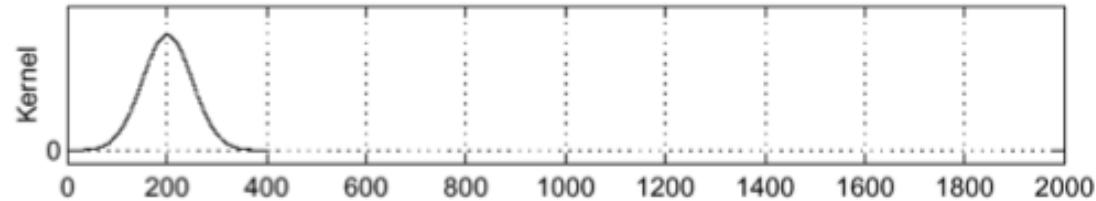
Differentiation is very sensitive to noise

When using derivative filters, it is critical to blur first!

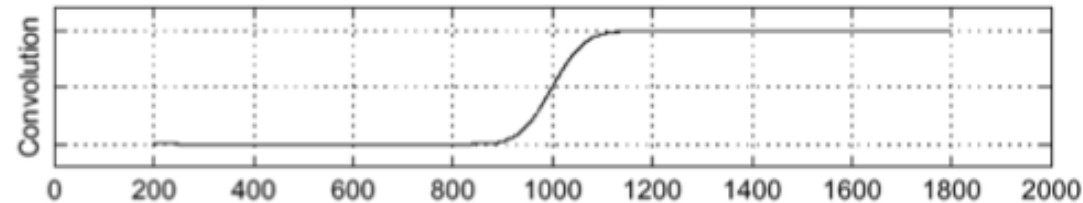
f input



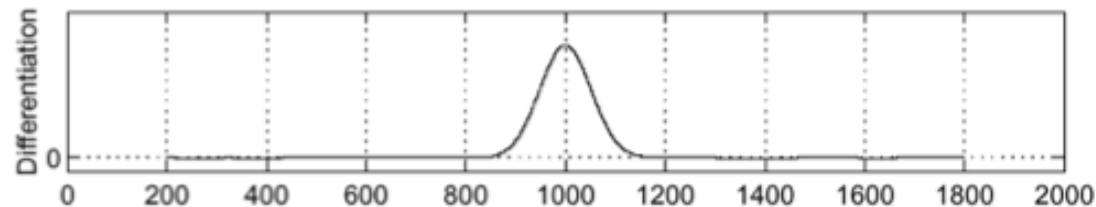
h Gaussian



$h * f$ blurred



$\frac{\partial}{\partial x}(h * f)$ derivative of blurred



Where is the edge?

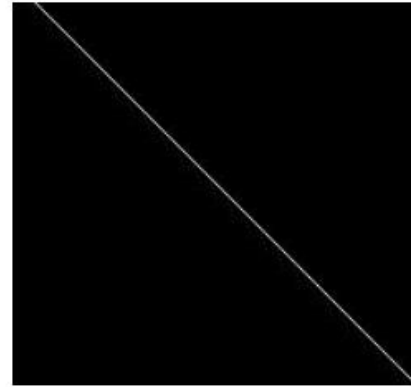
Look for peaks in

$$\frac{\partial}{\partial x}(h * f)$$

Trade Off Noise vs. Edge Localization



Image with Edge



Edge Location

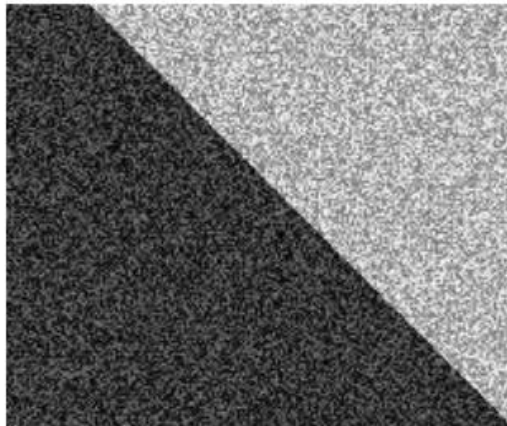
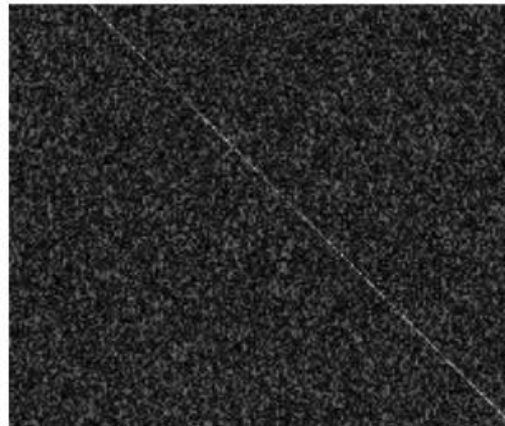
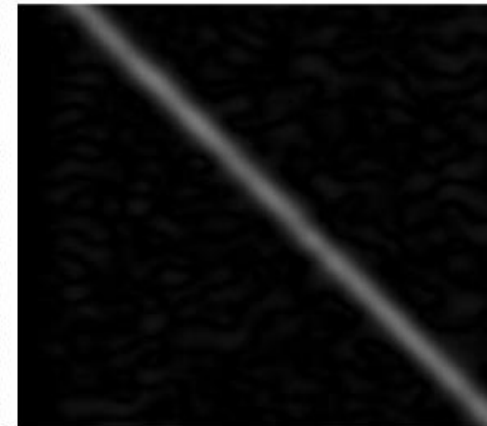


Image + Noise



W/out smoothing, edge and noise is picked up by derivative



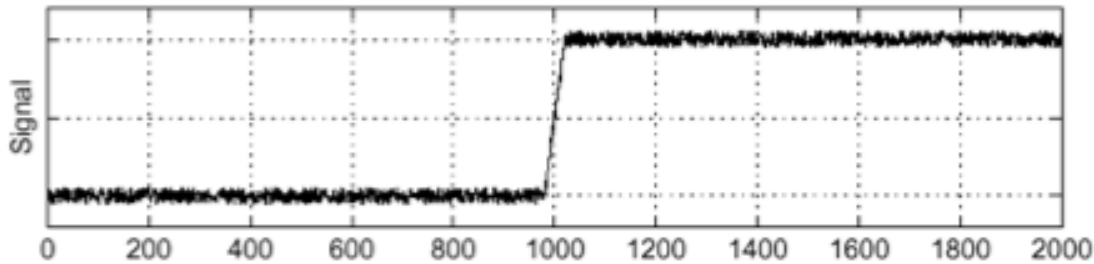
Smoothing removes the noise, but also blurs the edge

Derivative of Gaussian Filter

Differentiation Property of Convolution: $\frac{\partial}{\partial x}(h * f) = (\frac{\partial}{\partial x}h) * f$

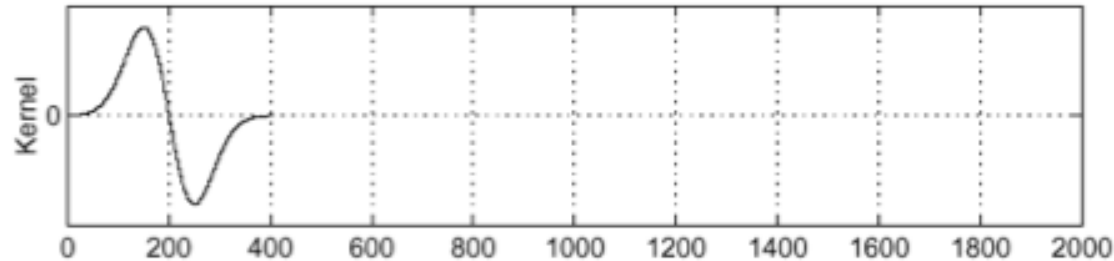
f

input



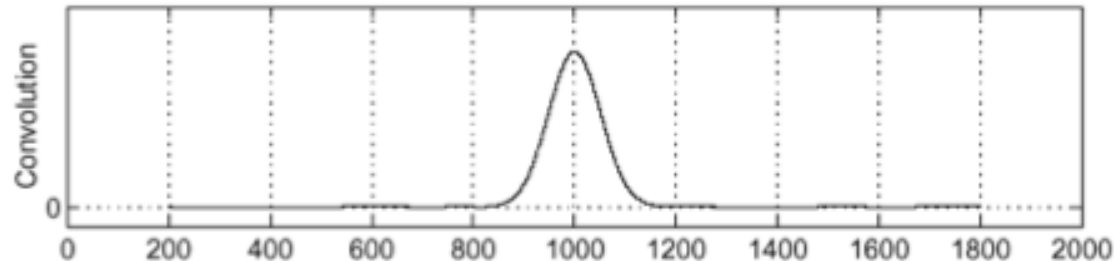
$\frac{\partial}{\partial x}h$

derivative of
Gaussian



$(\frac{\partial}{\partial x}h) * f$

output (same as
before)

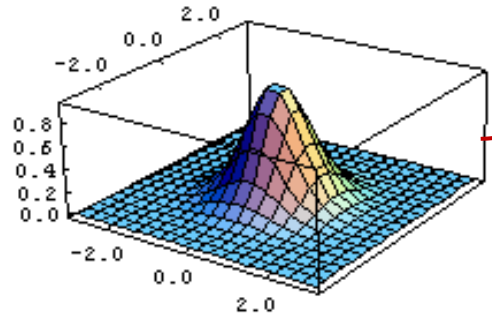


Derivative of Gaussian Filter

Differentiation is a convolution operation.

$$\frac{\partial}{\partial x} (h * f) = \left(\frac{\partial}{\partial x} h \right) * f = (h * g) * f$$

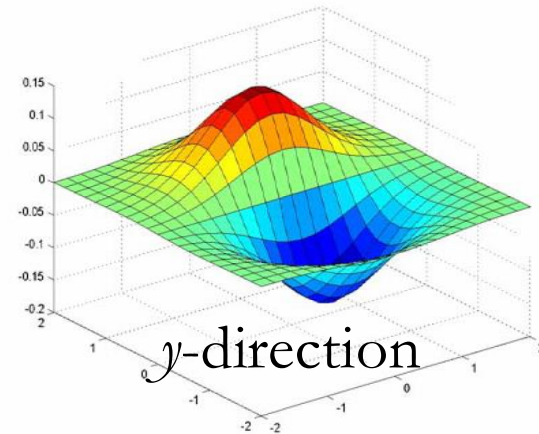
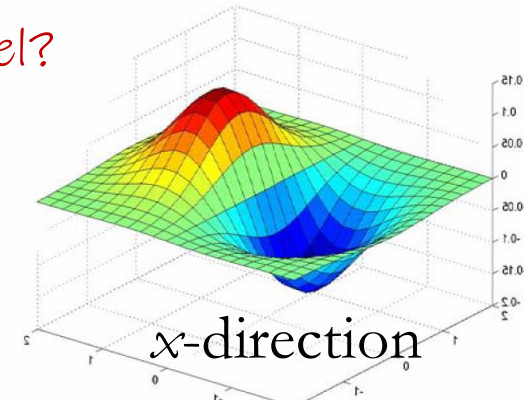
Which kernel?



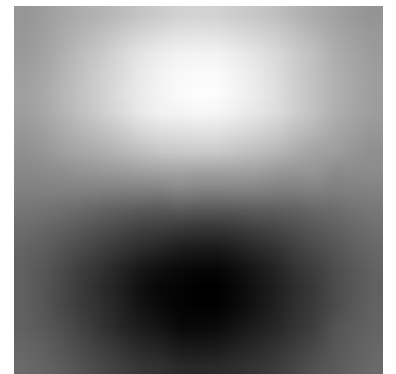
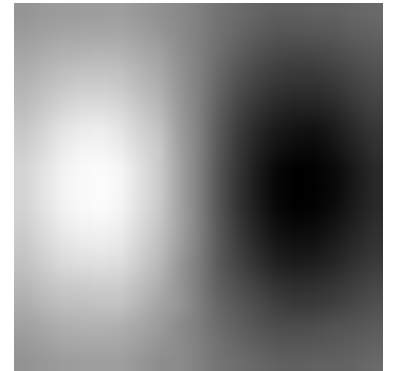
*

$$\begin{bmatrix} 1 & 0 & -1 \end{bmatrix}$$

=

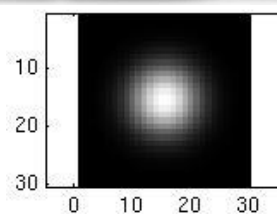
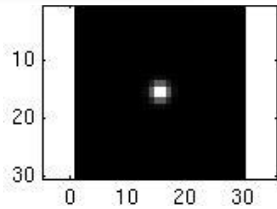


What should we set as σ for the Gaussian?

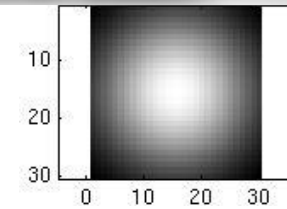


Smoothing with a Gaussian

σ is the “scale” of the Gaussian kernel and determines the extent of smoothing.



...



Effect of σ on Derivatives

Depending on the applied σ , different structures appear. Larger σ detects larger-scale edges, smaller σ detects finer edges.

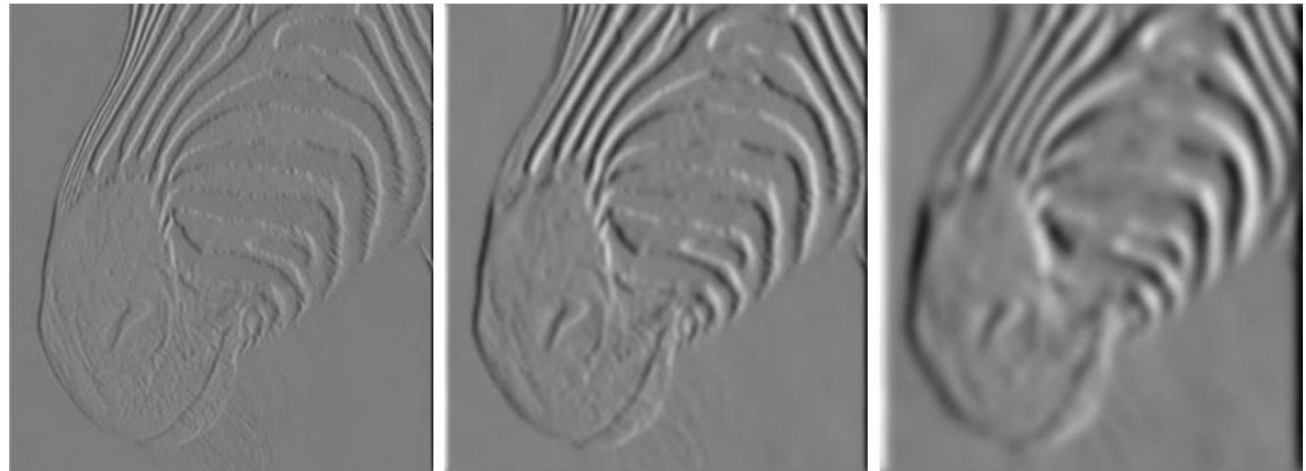
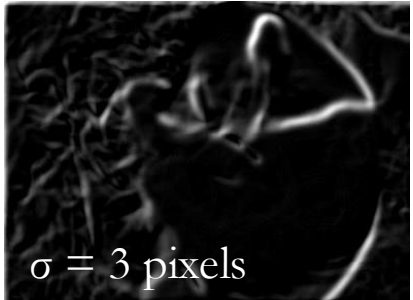
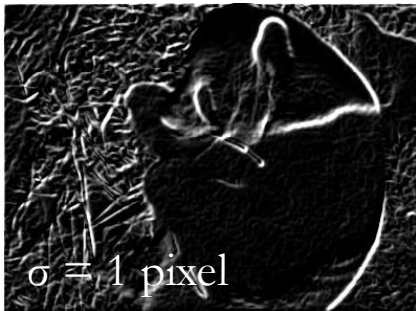
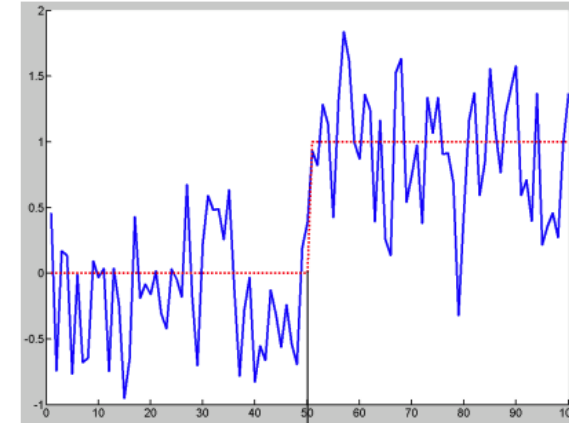
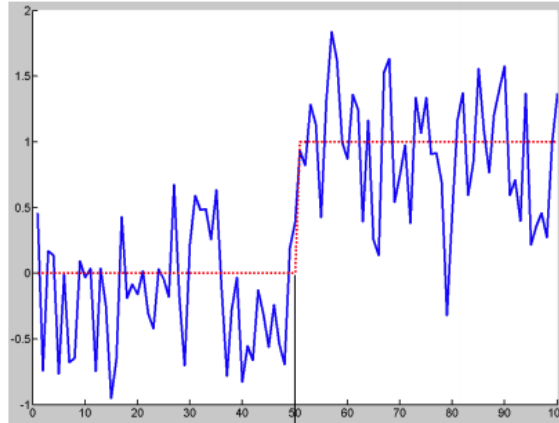


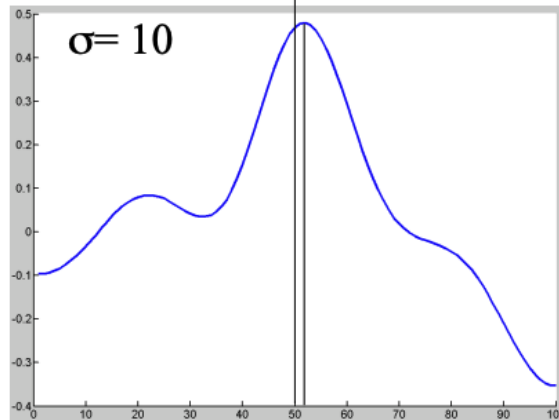
FIGURE 5.3: The scale (i.e., σ) of the Gaussian used in a derivative of Gaussian filter has significant effects on the results. The three images show estimates of the derivative in the x direction of an image of the head of a zebra obtained using a derivative of Gaussian filter with σ one pixel, three pixels, and seven pixels (**left to right**). Note how images at a finer scale show some hair, the animal's whiskers disappear at a medium scale, and the fine stripes at the top of the muzzle disappear at the coarser scale.

Which σ Should We Use?

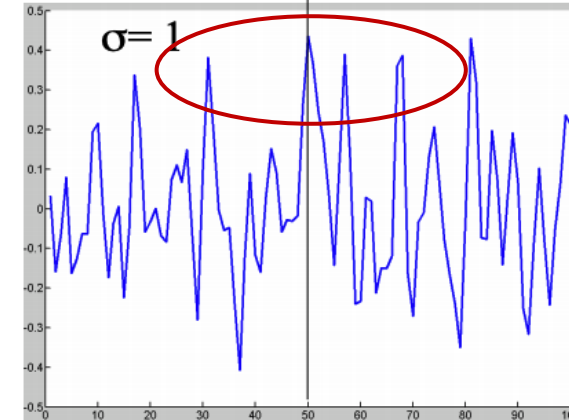
— true signal
— noisy observation



$\frac{\partial}{\partial x}(h * f)$ derivative of
smoothed signal



Large $\sigma \rightarrow$ Good detection (high SNR)
Poor localization



Small $\sigma \rightarrow$ Poor detection (low SNR)
Good localization

Gets difficult to
find the correct
maximum
corresponding to
the edge

Laplace filter

A second derivative filter that can also be approximated with finite differences.

first-order
finite difference

$$f'(x) = \lim_{h \rightarrow 0} \frac{f(x + 0.5h) - f(x - 0.5h)}{h}$$



1D derivative filter

1	0	-1
---	---	----

second-order
finite difference

$$f''(x) = \lim_{h \rightarrow 0} \frac{f(x + h) - 2f(x) + f(x - h)}{h^2}$$



1D Laplace filter

1	-2	1
---	----	---

Discrete
Laplacian
of x,y

$$\begin{aligned} \nabla^2 f(x, y) = & f(x + 1, y) + f(x - 1, y) \\ & + f(x, y + 1) + f(x, y - 1) - 4f(x, y) \end{aligned}$$



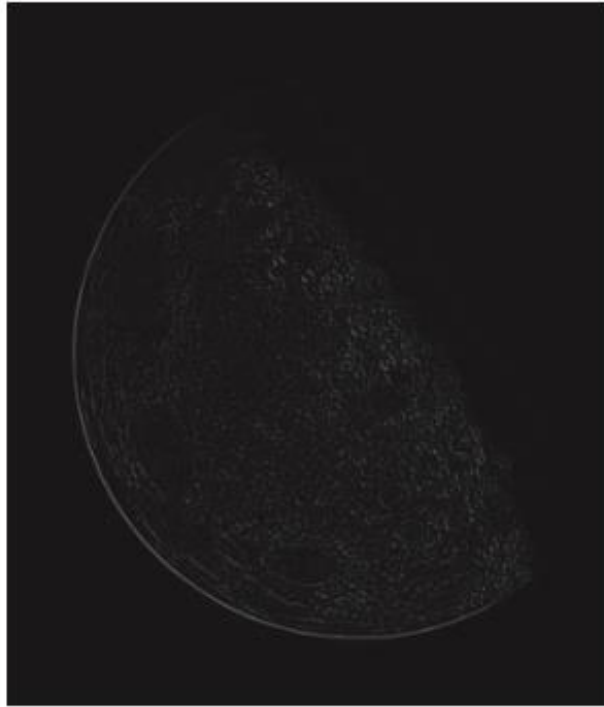
2D Laplace filter

0	1	0
1	-4	1
0	1	0

Sharpening with Laplacians



A: original



B: filtered with

Derivative filter:
highlights transitions.

0	1	0
1	-4	1
0	1	0



A-B: sharpening effect

Add original image to sharpened
details. Use a subtraction sign
because centre coefficient of
kernel (-4) is negative!



A-filtering with

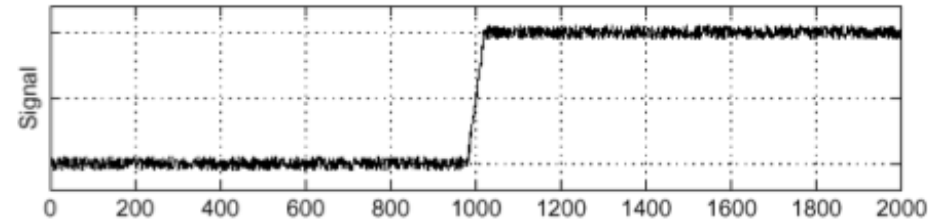
Extension of
Laplacian kernel with
diagonal terms.

1	1	1
1	-8	1
1	1	1

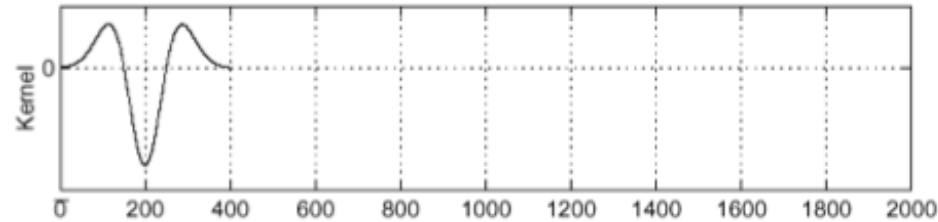
Laplacian of Gaussian (LoG) filter

As with derivative, we can combine Laplace filtering with Gaussian filtering

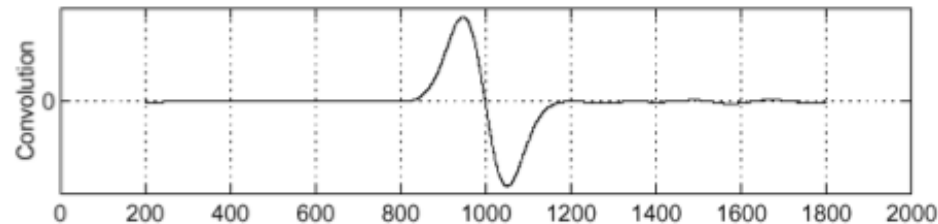
input



Laplacian of
Gaussian

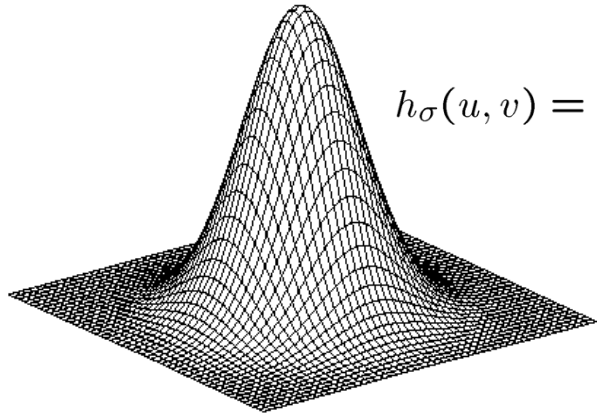


output



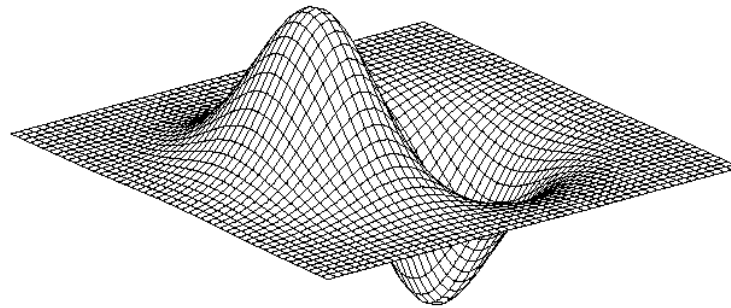
“zero crossings” at edges

2D Gaussian filters



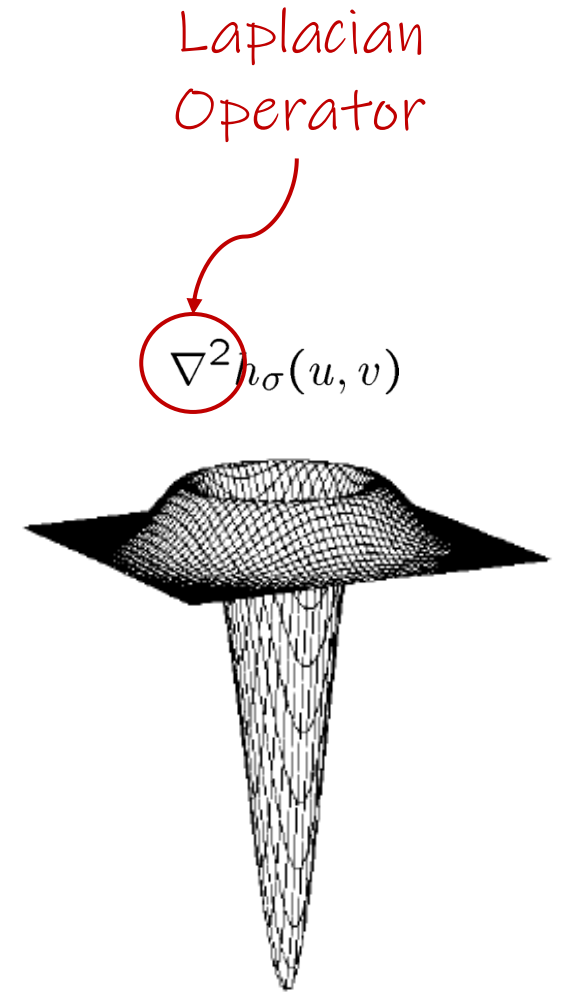
$$h_{\sigma}(u, v) = \frac{1}{2\pi\sigma^2} e^{-\frac{u^2+v^2}{2\sigma^2}}$$

Gaussian



$$\frac{\partial}{\partial x} h_{\sigma}(u, v)$$

Derivative of Gaussian



Laplacian
Operator

$$\nabla^2 h_{\sigma}(u, v)$$

Laplacian of Gaussian



Laplace and LoG filtering examples

Also sensitive to noise

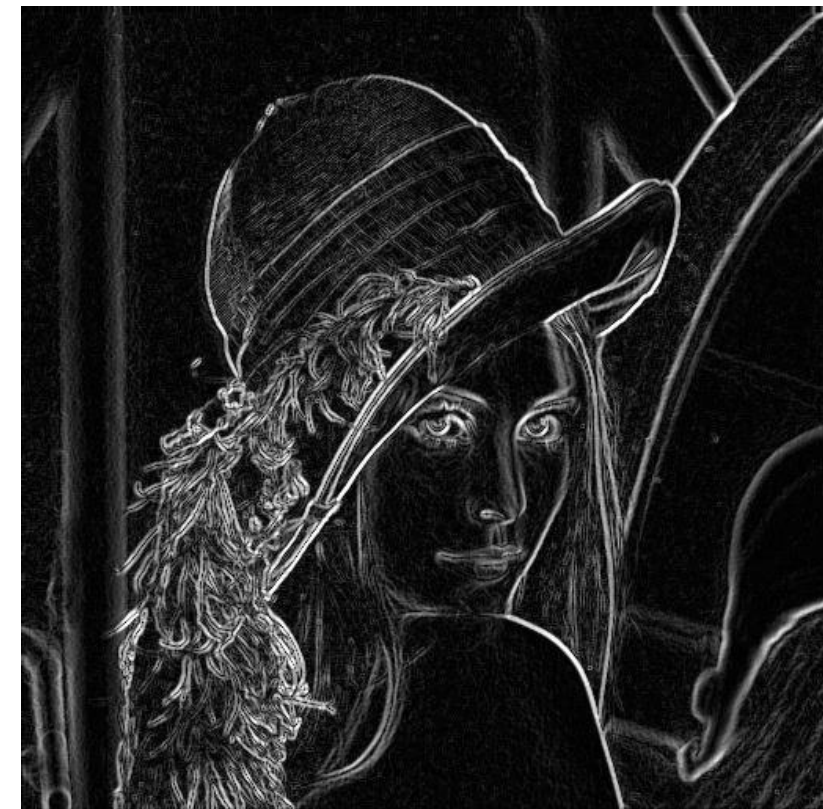


Laplace filtering

Smoothing w/ Gaussian helps

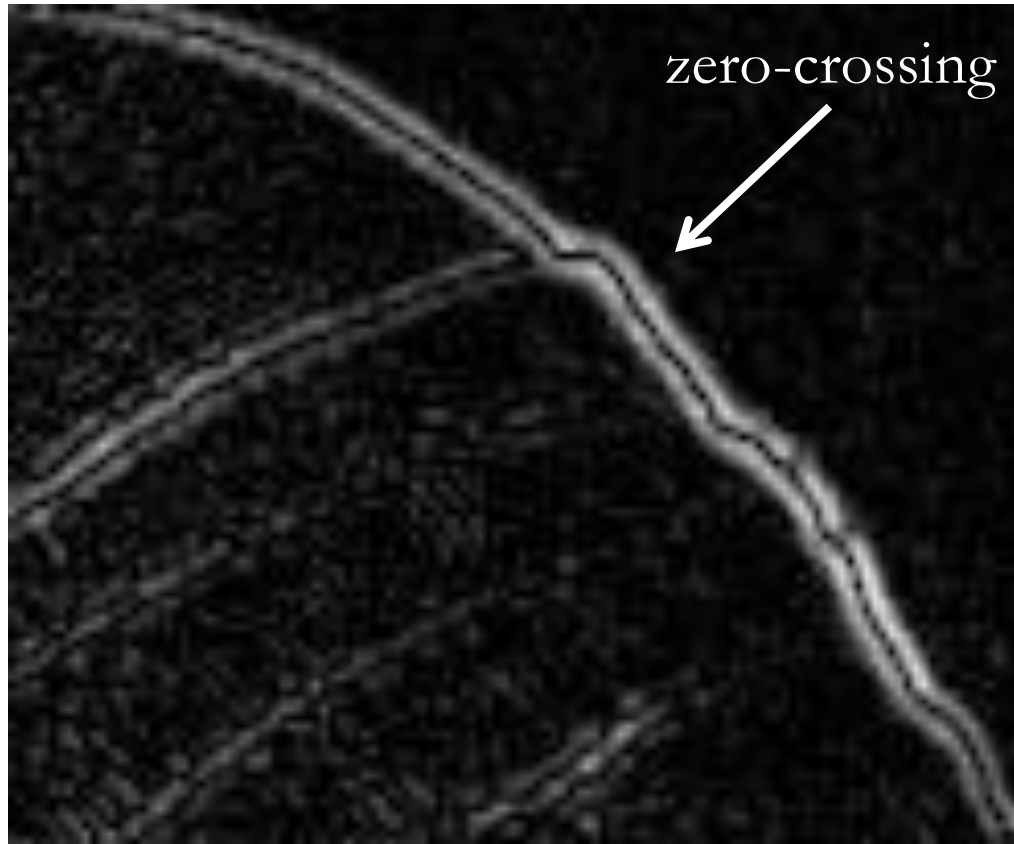


Laplacian of Gaussian filtering

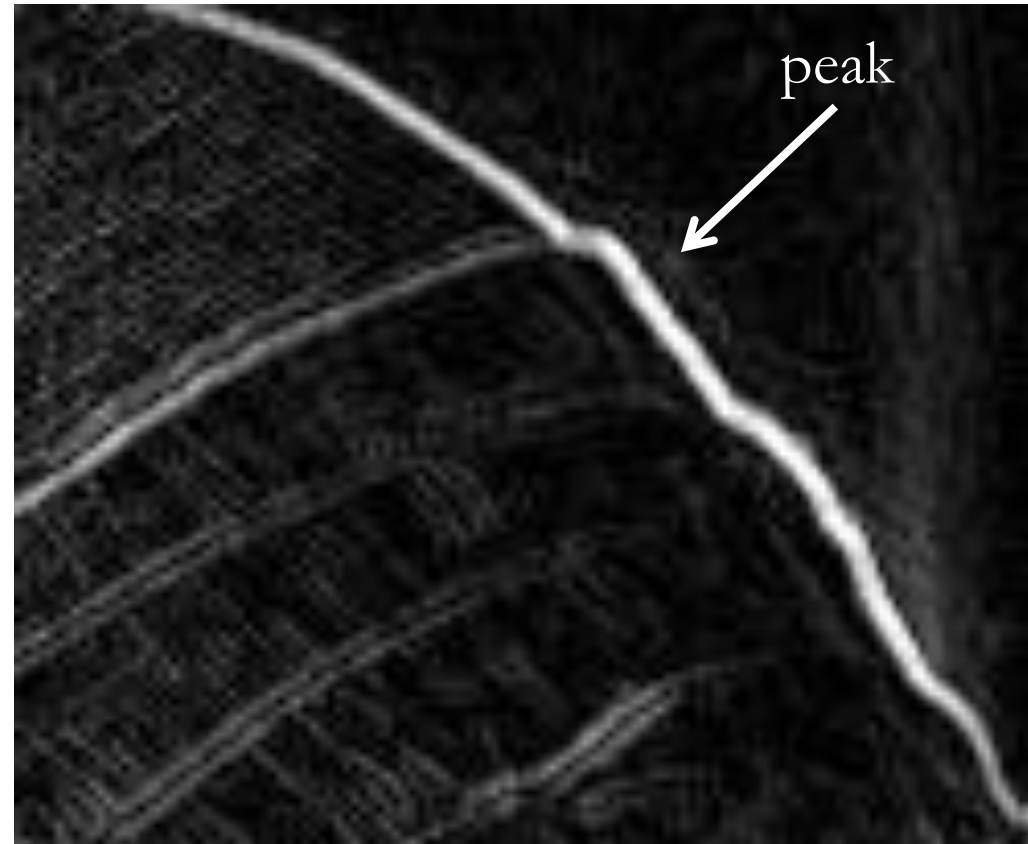


Derivative of Gaussian filtering

Laplacian of Gaussian vs Derivative of Gaussian



Laplacian of Gaussian filtering



Derivative of Gaussian filtering

Zero crossings localize edges more accurately but are less convenient to use.

In-Lecture Question:



What might be a good reason to smooth an image before applying a derivative filter?

- A. Derivatives suppress noise automatically
- B. Derivatives amplify high-frequency noise
- C. Smoothing increases spatial detail
- D. Smoothing makes the gradient zero everywhere

A Computational Approach to Edge Detection

JOHN CANNY, MEMBER, IEEE

Abstract—This paper describes a computational approach to edge detection. The success of the approach depends on the definition of a *comprehensive* set of goals for the computation of edge points. These goals must be precise enough to delimit the desired behavior of the detector while making minimal assumptions about the form of the solution. We define detection and localization criteria for a class of edges,

detector as input to a program which could isolate simple geometric solids. More recently the model-based vision system ACRONYM [3] used an edge detector as the front end to a sophisticated recognition program. Shape from motion [29], [13] can be used to infer the structure of

Canny Edge Detector

Adds non-maximum suppression and hysteresis thresholding to find edges from gradient outputs.

Gradients to Edges



original*
image



gradient
magnitude



gradient magnitude
after thresholding

How to turn these thick
regions of the gradient
into single-pixel edges?



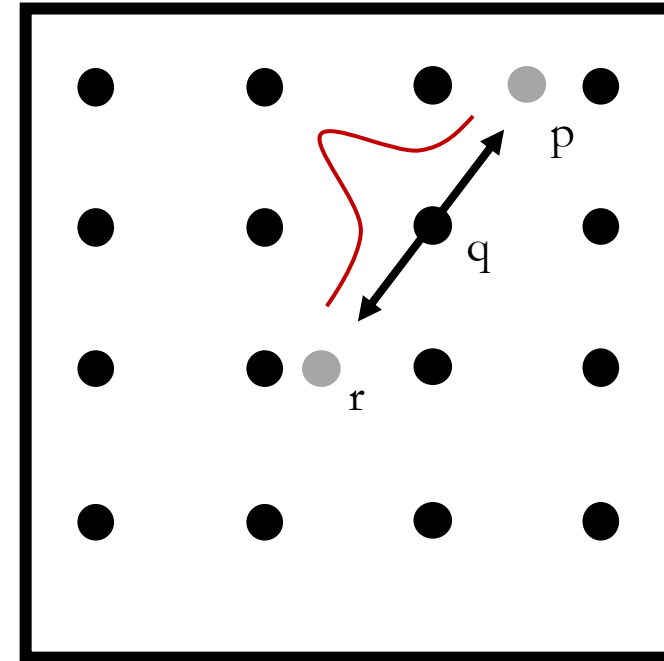
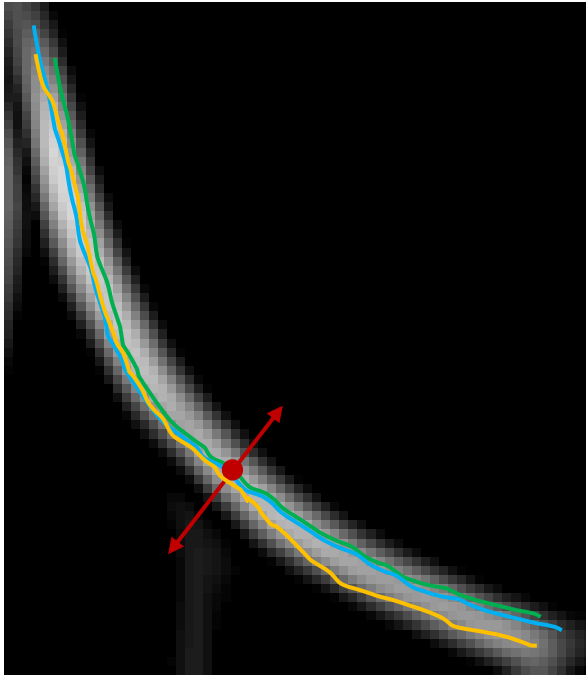
edge image

* Smoothing to suppress noise. Increase contrast to enhance edges.

Canny Edge Detector

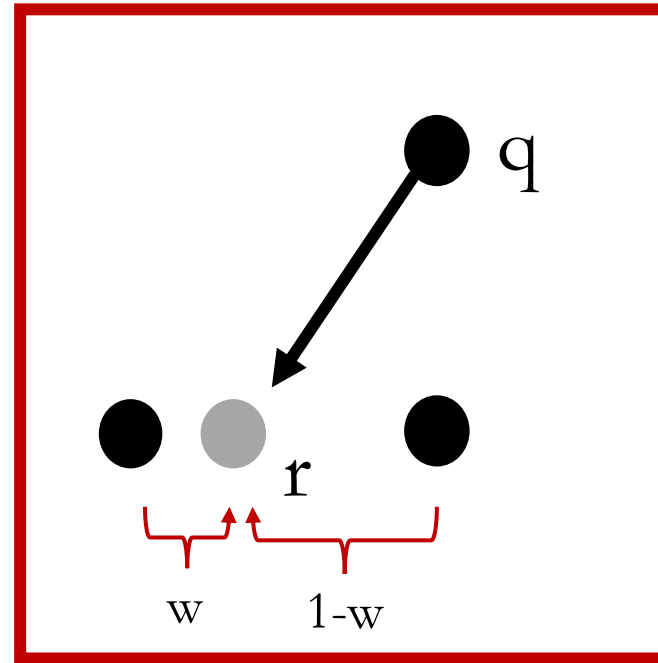
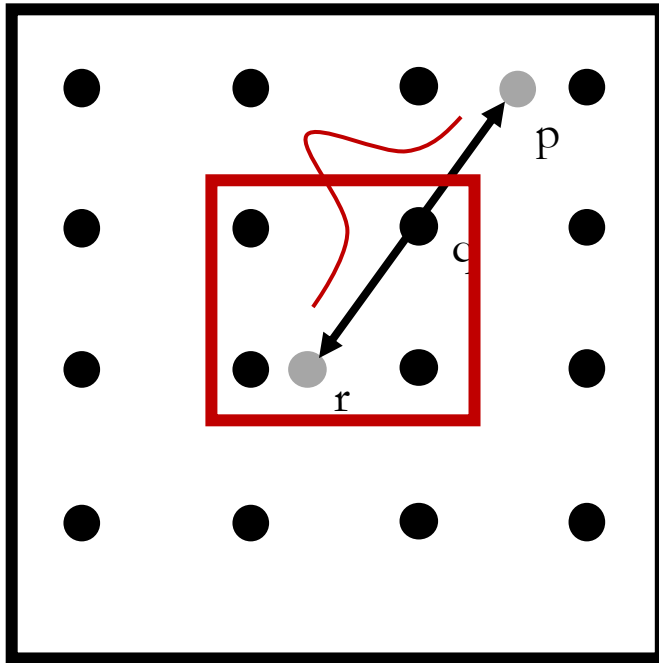
1. Filter image with derivative of Gaussian
2. Find magnitude and orientation of gradient
3. Non-maximum suppression
 - Thin wide “ridges” down to single pixel width
4. Hysteresis thresholding & linking:
 - Creates continuous edges where possible

Edge Thinning via Non-Maximum Suppression



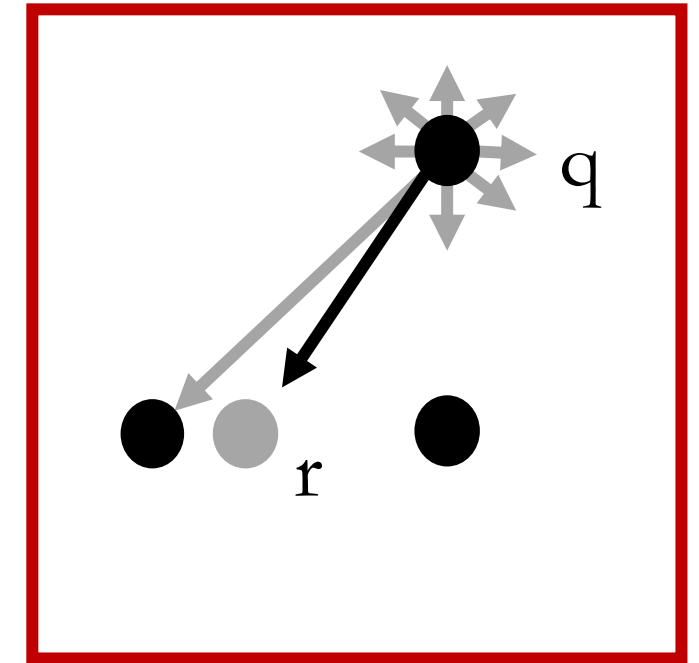
For each pixel, check if there is a local max along the gradient (i.e. across the edge, based on the gradient orientation). Only local max pixels are kept as edges.

Edge Thinning via Non-Maximum Suppression



Interpolation

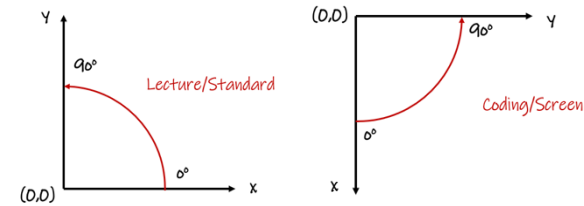
Estimate by interpolating
between the closest pixels.



Approximation

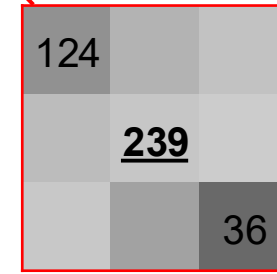
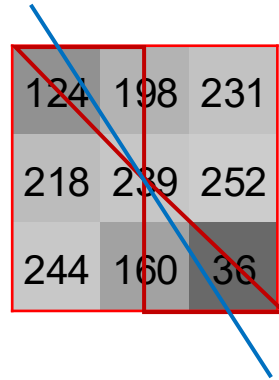
Estimate by closest pixel.
(Read-out is faster).

Non-Maximum Suppression Example



Magnitude

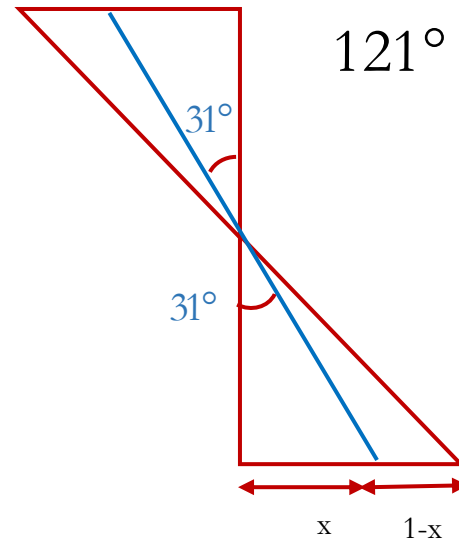
2	5	5	6	6	7	5	7
6	5	4	18	25	26	17	10
1	3	31	124	198	231	219	198
2	9	78	218	239	252	264	270
7	14	104	244	160	36	53	72
6	14	119	244	142	13	10	13
6	16	135	242	124	6	6	10
7	18	149	242	110	2	9	4



No interpolation approximation

Orientation

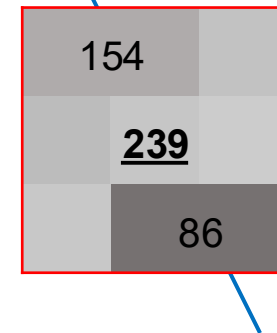
90	79	79	121	90	45	79	124
108	63	90	131	97	67	80	90
-135	-18	138	128	108	86	86	86
-180	117	151	148	121	88	85	85
135	146	169	172	168	96	79	81
162	172	176	177	179	-135	-90	-72
-180	173	175	175	175	-180	-59	-53
146	167	176	175	173	90	-27	-45



$$x = \tan\left(\frac{31}{180} \cdot \pi\right); x \approx 0.6$$

$$0.6 \cdot (124 - 198) + 198 \approx 154$$

$$0.6 \cdot (36 - 160) + 160 \approx 86$$



Keep as part of edge

Discontinuous Edges

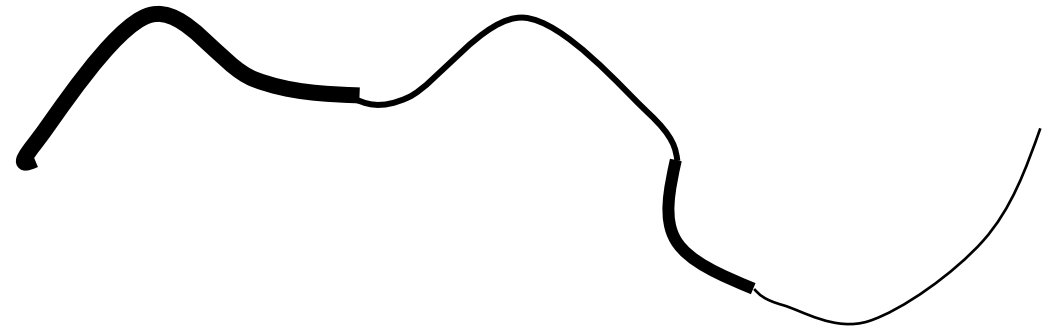


gradient magnitude
after thresholding



edge image

*Problem: pixels along this edge
didn't survive the thresholding*



Hysteresis thresholding: use a high threshold to start edge curves, and a low threshold to continue growing them.

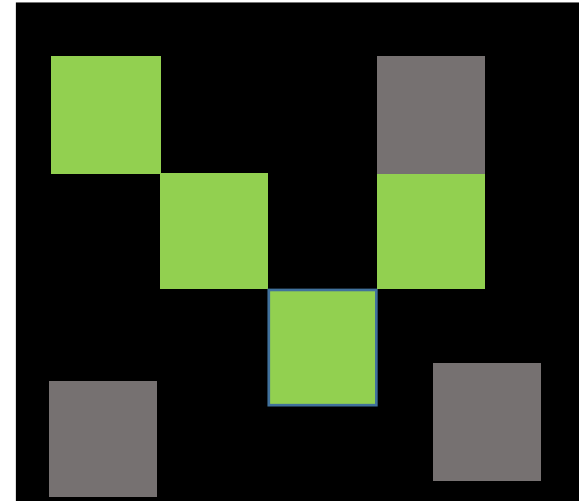
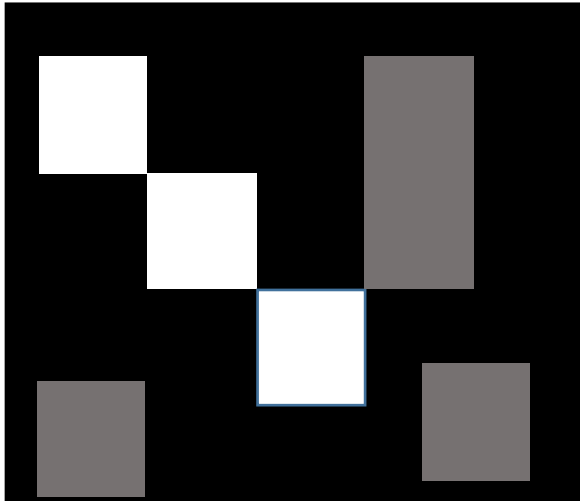
Hysteresis Thresholding

White: Clears high threshold

Grey: clears low threshold but not high threshold

Mark pixels that clear high threshold as boundary

If there are pixels that clear low threshold and are connected to a boundary, mark them as boundary too



Hysteresis Thresholding

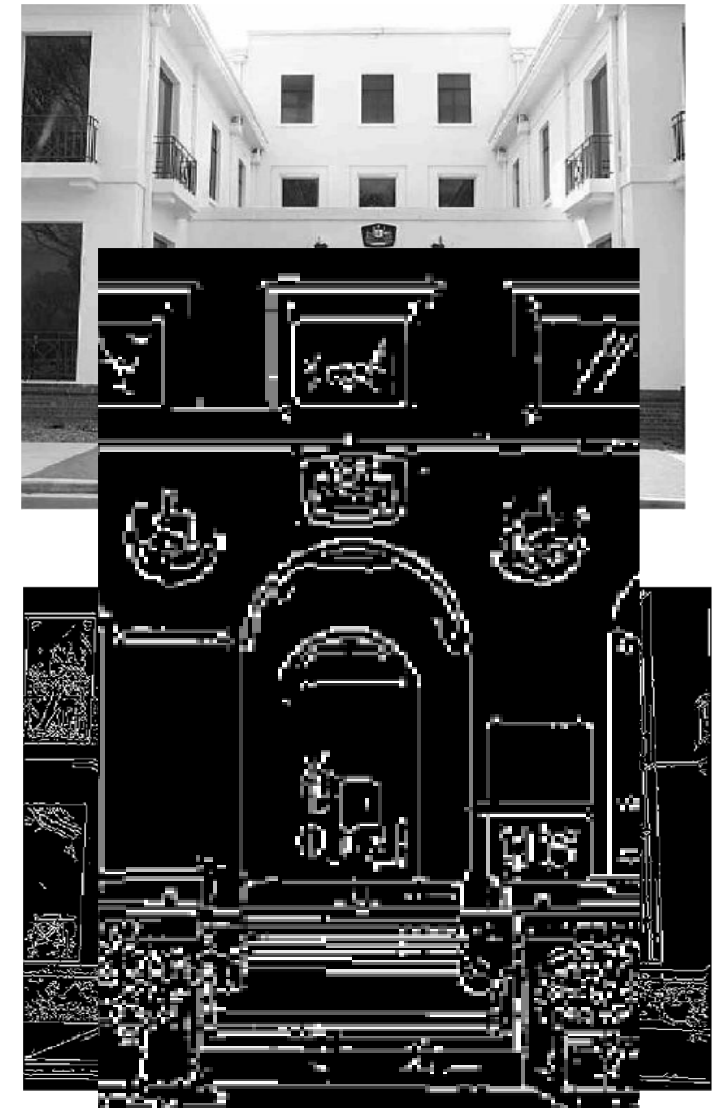


high threshold
(strong edges)



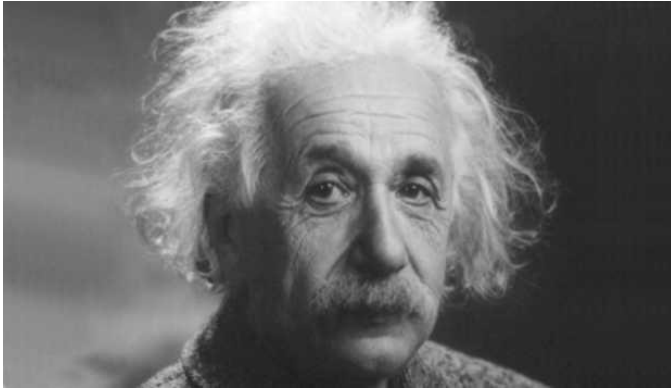
low threshold
(weak edges)

03. Gradients & Edges

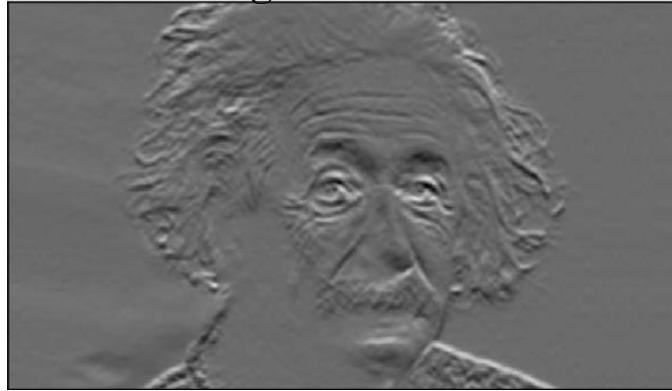


hysteresis threshold
(linked edges)

Canny Step-by-Step



1: Filter image with derivative of Gaussian

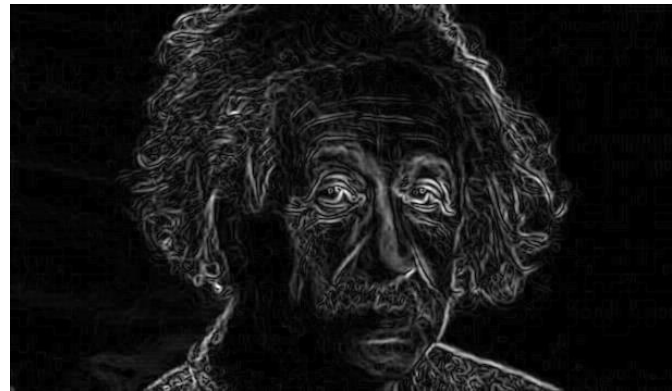


gradients in y-dir



gradients in x-dir

2. Compute gradient magnitude and orientation



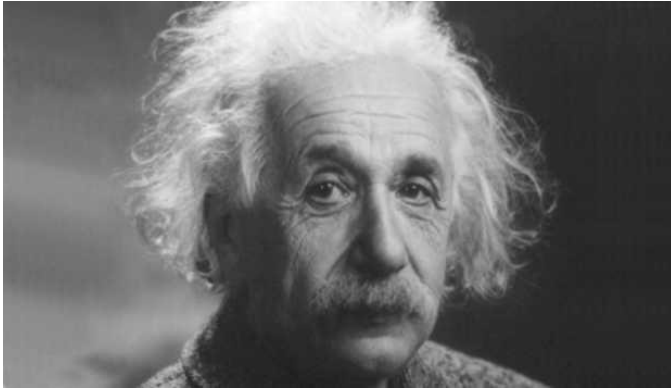
gradient magnitude



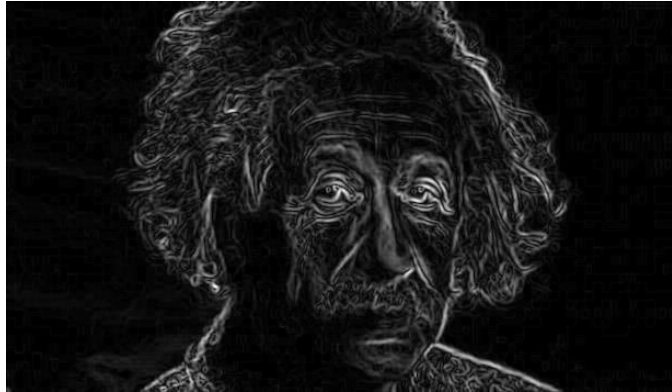
(visualization of) gradient
orientation



Canny Step-by-Step



3. Non-Maximum Suppression



G: gradient magnitude, [0-255]



E: Single-pixel wide gradient image, [0-255]

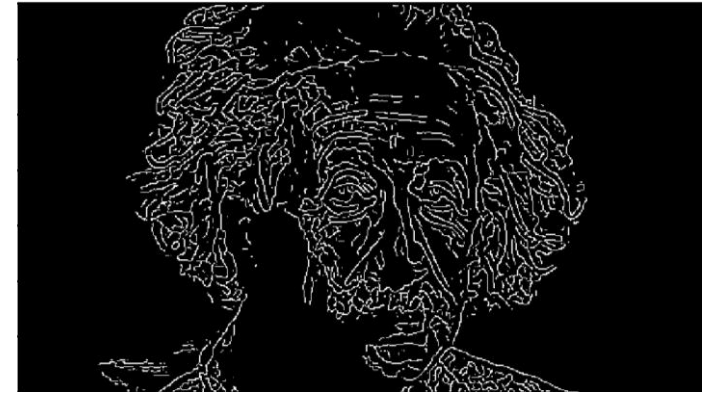
4. Hysteresis Thresholding & Linking



Final Canny Edge Image



E_H : high threshold, 0 or 255
(keeps only most confident pixels)



E_L : low threshold, 0 or 255
(produces spurious edges)

Linking

Find image gradients

- G: gradient image, has values between 0 to 255

Apply Non-Maximum Suppression

- E is single pixel gradient image after NMS, also has values between 0 to 255

Thresholding

- Apply high threshold H to E to get E_H (all values passing threshold set to 255, all fail set to 0)
- Apply low threshold L to E to get E_L (all values passing threshold set to 255, all fail set to 0)

Linking

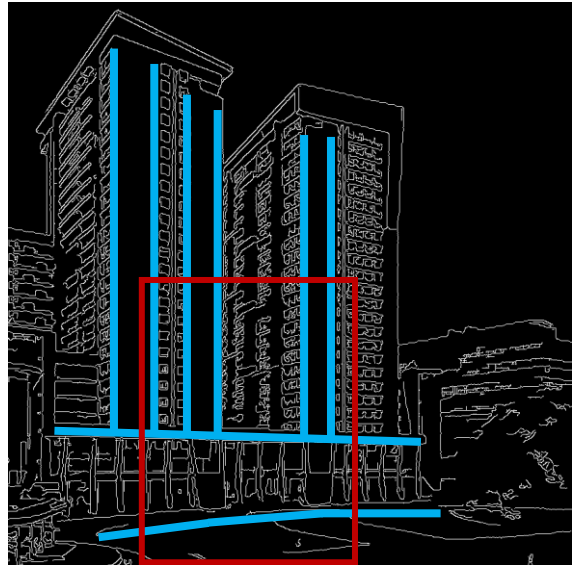
- $D = E_L - E_H$, keep only edges of D which are linked or connected to $E_H \rightarrow D'$
- Composite E_H and D' to get final edge image

Lines & Hough Transform

Why fit straight lines?
Mathematics of lines, Hough Transform

Motivation: Fitting (Straight) Lines

Many objects characterized by presence of straight lines.



Q: Doesn't edge detection already give us the lines?

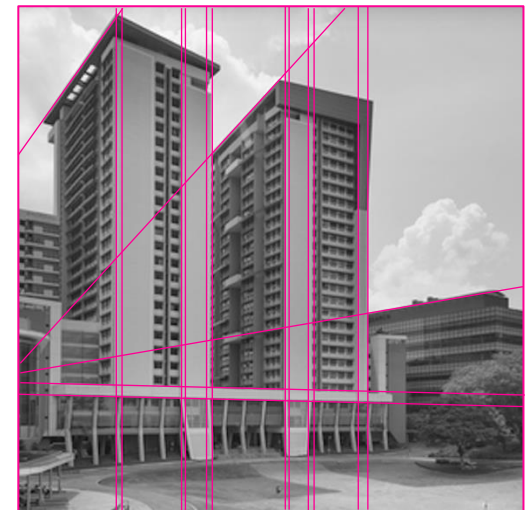
A: Canny gives us *all* edges (lines, curves, noise, etc.)

Fitting Lines

- Which points belong to which line?
- What about missing edge points and noise?

Hough Transform for Line Fitting

- Given points that belong to a line, what is the line?
- How many lines are there?
- Which points belong to which lines?
- **Hough Transform** is a voting technique to answer all of these questions.
 - For each edge point, record a vote for each possible line that passes through that point
 - Look for lines which receive many votes.



Equation of a Line: Slope Intercept Form

$$y = mx + b$$



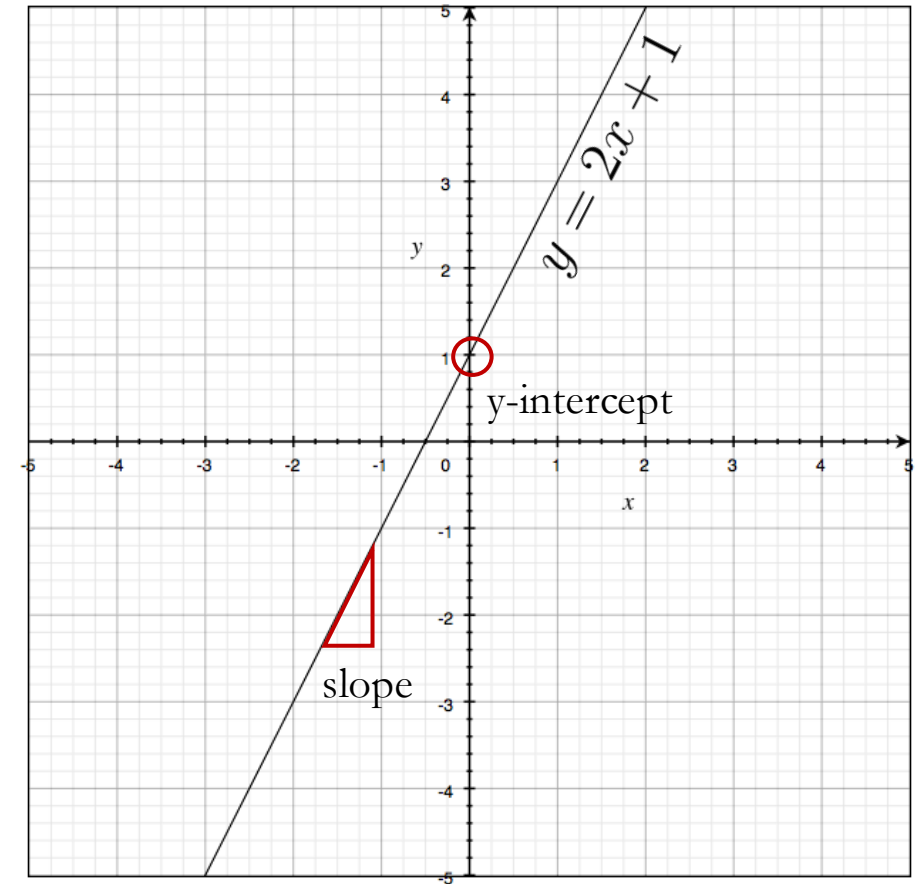
slope



y-intercept

$$y = 2x + 1$$

We switch to Cartesian coordinates for more intuitive line mathematics.



Equation of a Line: Normal Form

$$x \cos \theta + y \sin \theta = \rho$$

angle
length

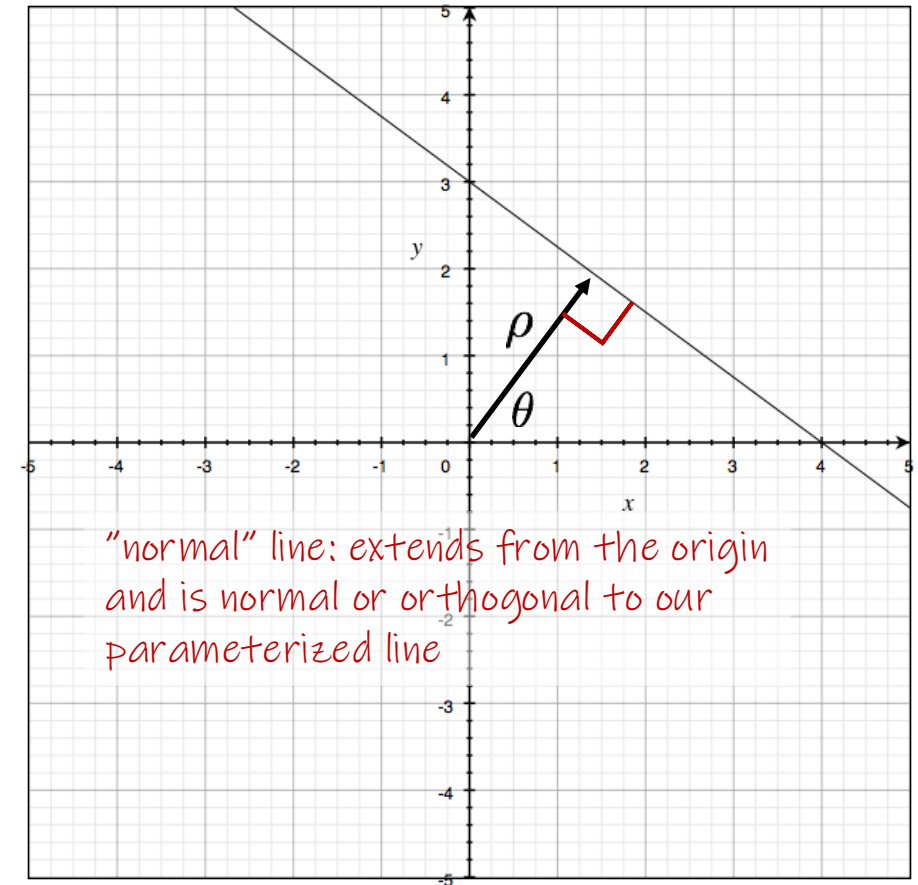
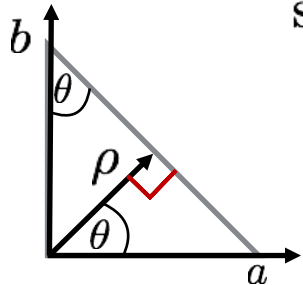
Derivation:

$$\cos \theta = \frac{\rho}{a} \rightarrow a = \frac{\rho}{\cos \theta}$$

$$\sin \theta = \frac{\rho}{b} \rightarrow b = \frac{\rho}{\sin \theta}$$

plug into: $\frac{x}{a} + \frac{y}{b} = 1$

$$x \cos \theta + y \sin \theta = \rho$$



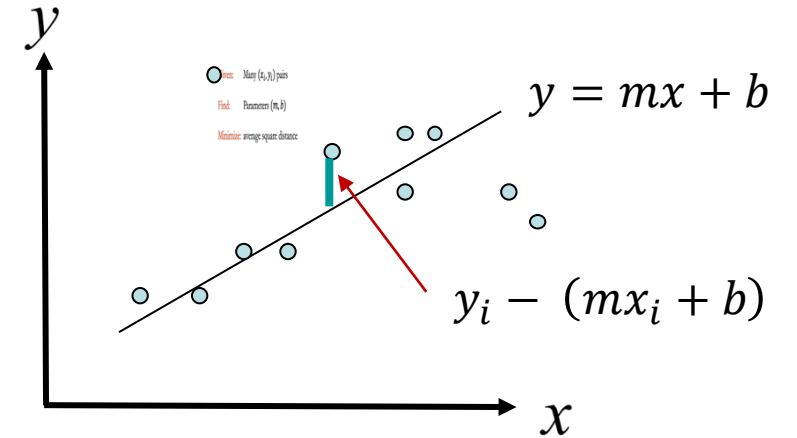
Simple Line Fitting

Given: Many (x_i, y_i) pairs
point index

Find: Parameters (m, b)

Minimize: average square distance $E = \frac{1}{N} \sum_i [y_i - (mx_i + b)]^2$ $(m, b) = \operatorname{argmin}_{m,b} E$
total number of points

Any problems with
this approach?



Problems with Simple Line Fitting

Where is the line that minimizes E?

$$E = \frac{1}{N} \sum_i [y_i - (mx_i + b)]^2$$

Square term heavily penalizes outliers

- Error E must be formulated carefully.
- Reduce the impact of outliers
 - Voting-based approach
 - RANSAC (later lecture)

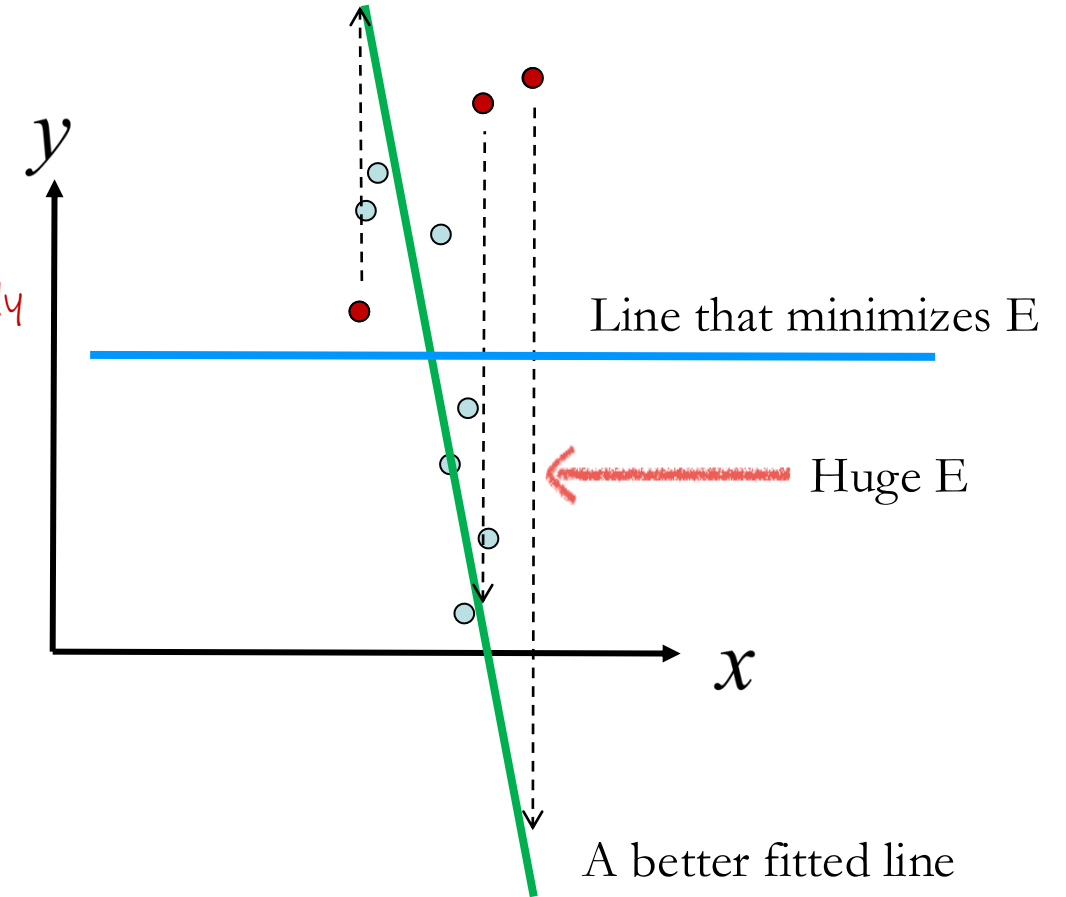


Image Space vs. Parameter Space

$$\begin{array}{ccc} \text{variables} & & \text{variables} \\ \swarrow \downarrow \searrow & \xrightarrow{b = (-x)m + y} & \swarrow \downarrow \searrow \\ y = mx + b & & y - mx = b \\ \swarrow \downarrow \searrow & & \swarrow \downarrow \searrow \\ \text{parameters} & & \text{parameters} \end{array}$$

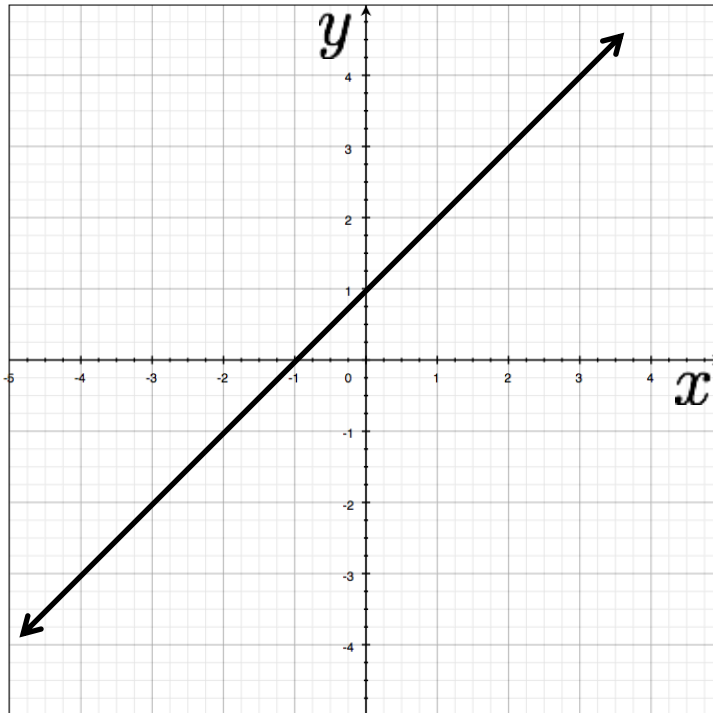
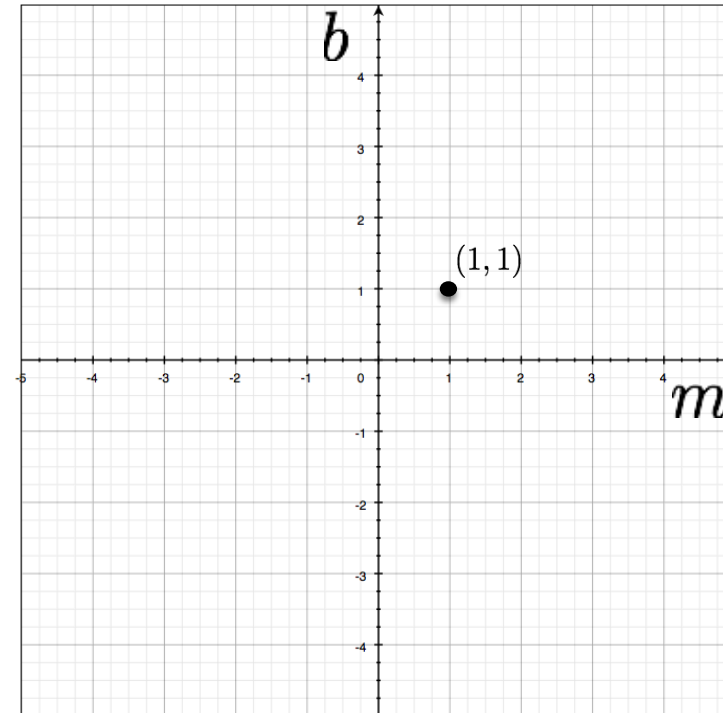


Image space

a line
becomes a
point



Parameter space

Image Space vs. Parameter Space

variables

$$y = mx + b$$

parameters

variables

$$y - mx = b$$

parameters

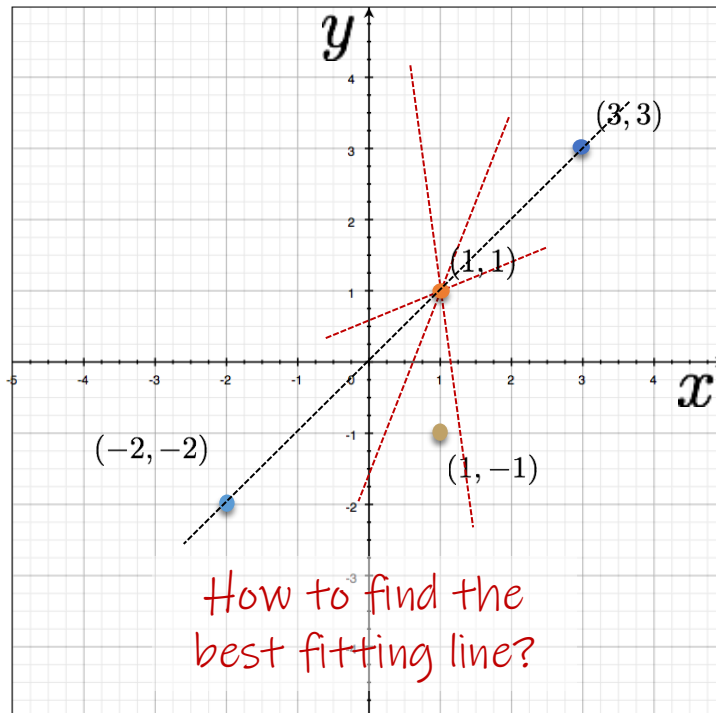
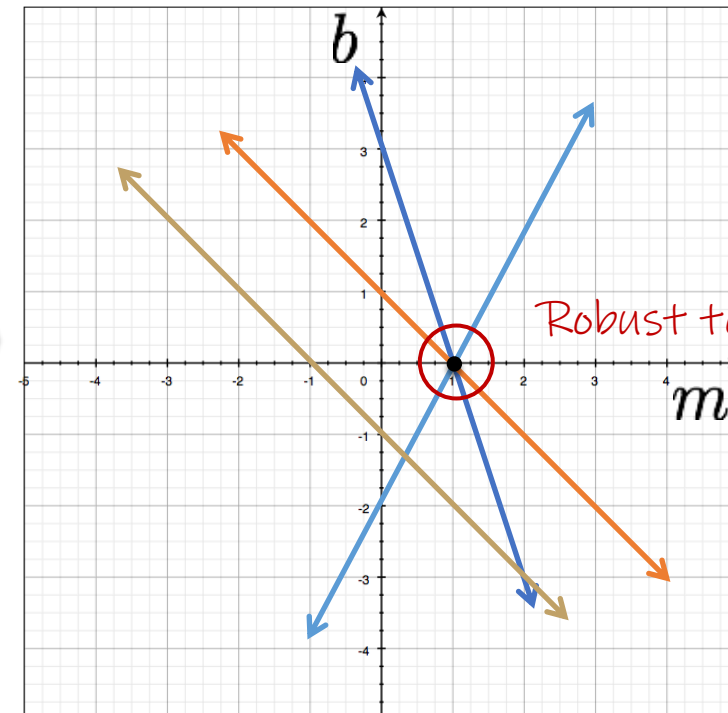


Image space

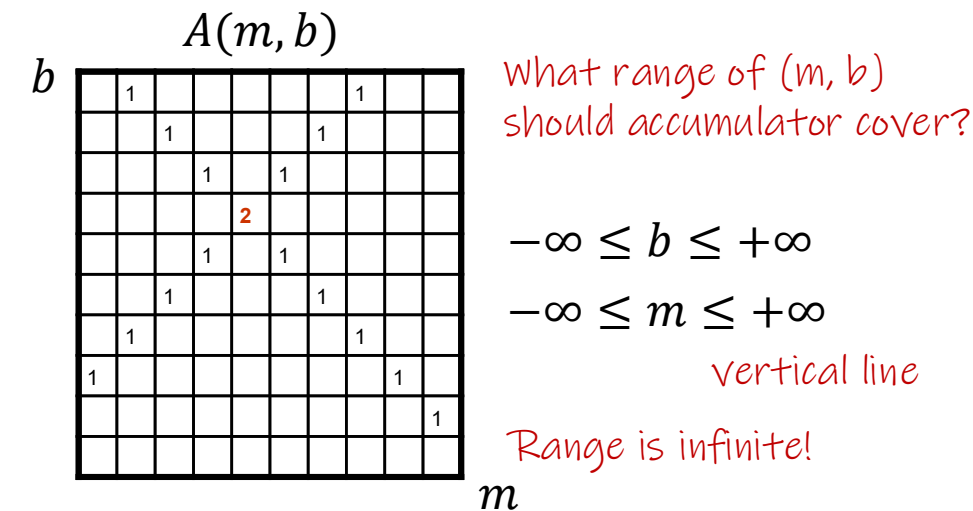
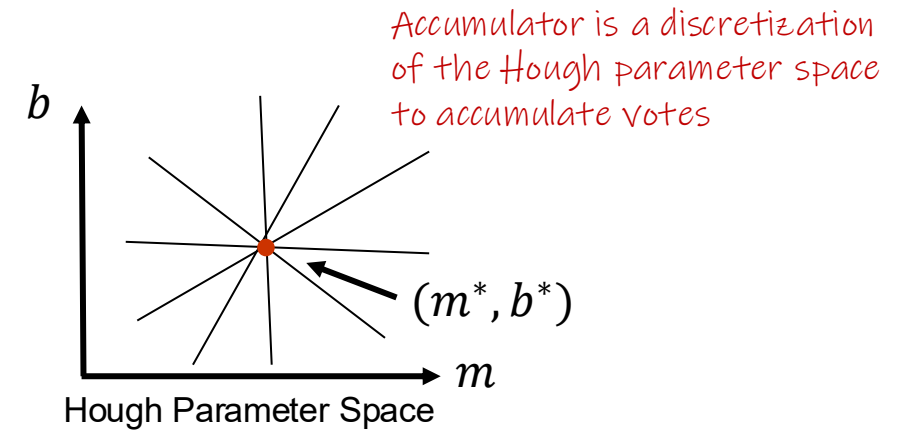


Parameter space

Line Detection with Hough Transform

Algorithm:

1. Quantize Parameter Space (m, b)
2. Create Accumulator Array $A(m, b)$
3. Set $A(m, b) = 0 \quad \forall m, b$
4. For each image edge point (x_i, y_i)
 For each element m
 Solve $b = x_i m + y_i$
 Increment $A(m, b) = A(m, b) + 1$
5. Threshold & find local maxima in $A(m, b)$
6. Detected line(s) given by $y = m^*x + b^*$



Alternative parameter space?

Normal Parameterization

variables

$$y = mx + b$$

parameters

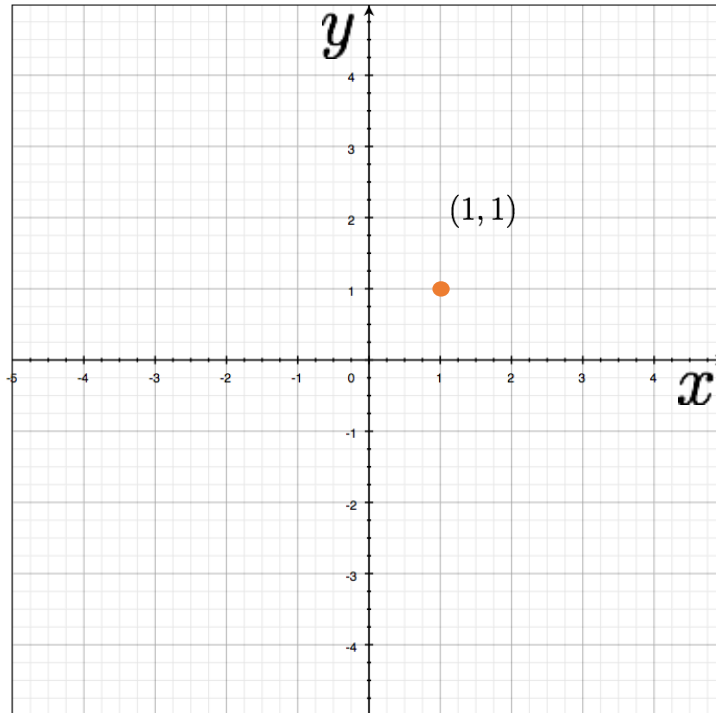
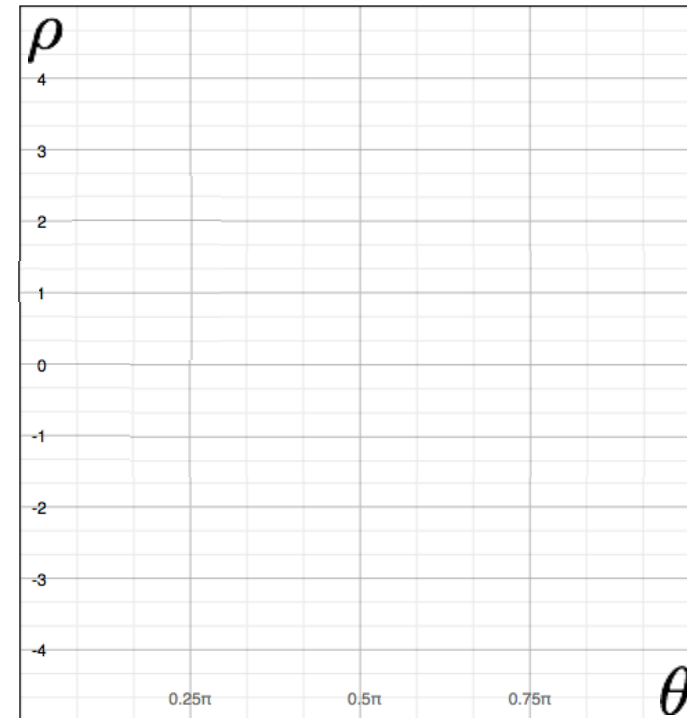


Image space

parameters

$$x \cos \theta + y \sin \theta = \rho$$

variables



Parameter space

a point
becomes?
a sinusoid

Finite range

$$0 \leq \theta \leq 2\pi$$

$$0 \leq \rho \leq \rho_{\max}$$

A Point to a Sinusoid

variables

$$y = mx + b$$

parameters

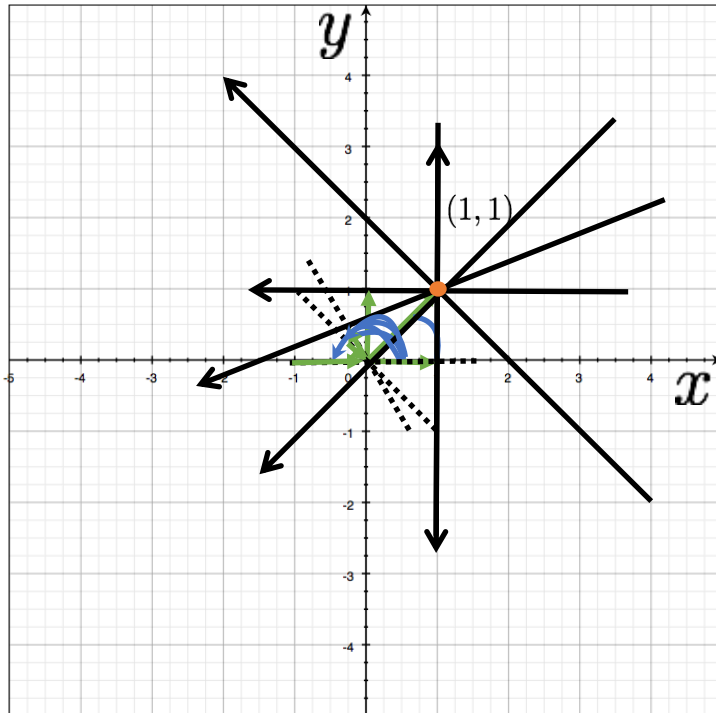


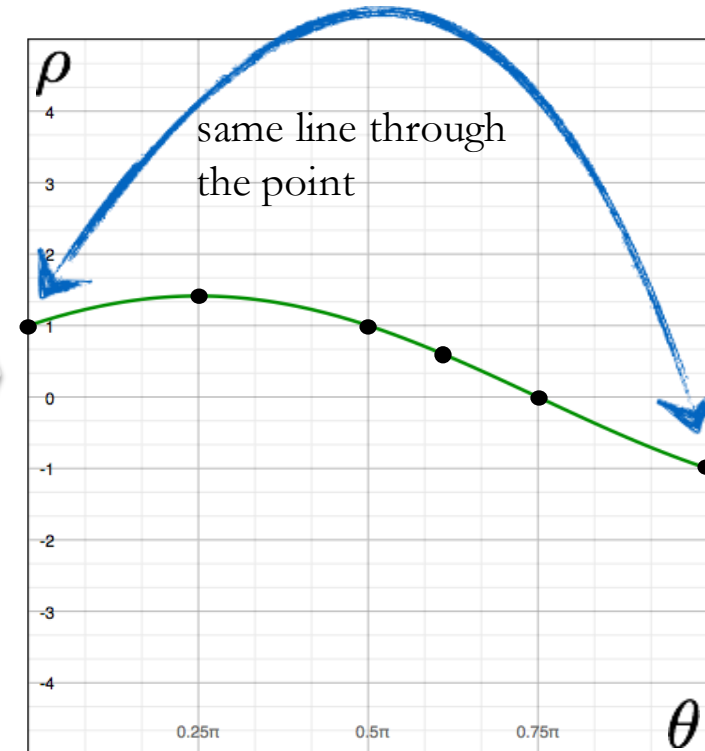
Image space

a point
becomes?
a sinusoid

parameters

$$x \cos \theta + y \sin \theta = \rho$$

variables



Parameter space

How can we have
a negative rho?

Image and (Normal) Parameter Space

variables

$$y = mx + b$$

parameters

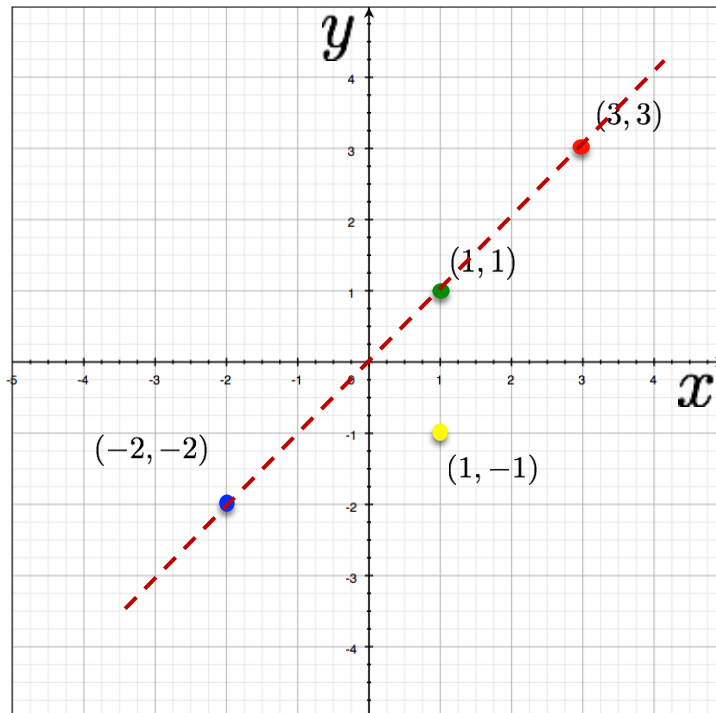
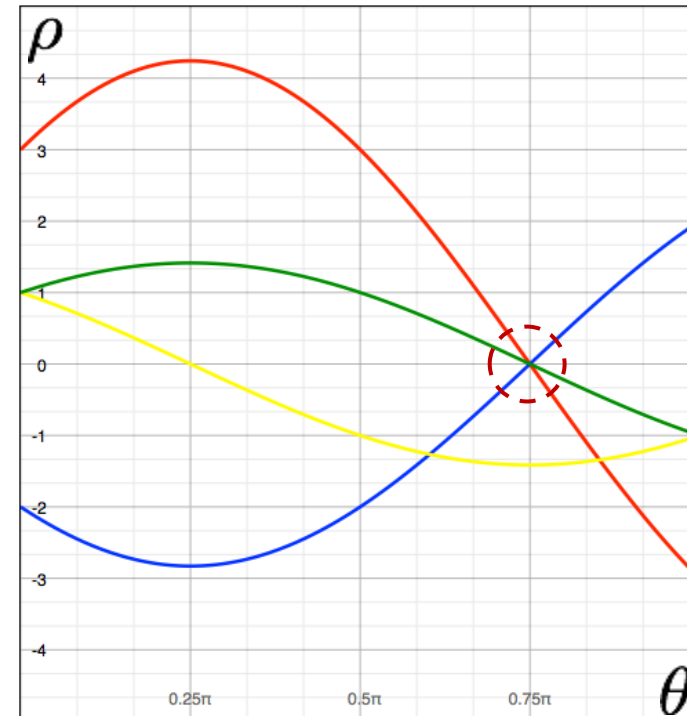


Image space

parameters

$$x \cos \theta + y \sin \theta = \rho$$

variables



Parameter space

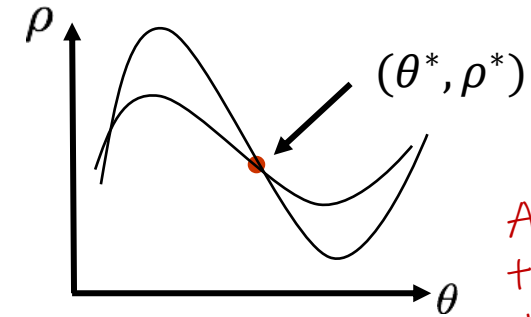
Points (on a line) become?
Intersecting
sinusoids

Normal Form

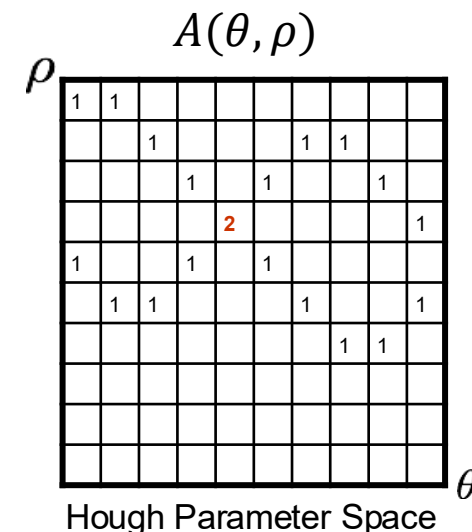
Line Detection with Hough Transform

Algorithm:

1. Quantize Parameter Space (θ, ρ)
2. Create Accumulator Array $A(\theta, \rho)$
3. Set $A(\theta, \rho) = 0 \quad \forall \theta, \rho$
4. For each image edge point (x_i, y_i)
For each element θ
Solve $\rho = x_i \cos \theta + y_i \sin \theta$
Increment $A(\theta, \rho) = A(\theta, \rho) + 1$
5. Threshold, find local maxima in $A(\theta, \rho)$
6. Detected line(s) given by $\rho^* = x \cos \theta^* + y \sin \theta^*$



All steps remain the same, just a change in the parameterization of the Hough space!



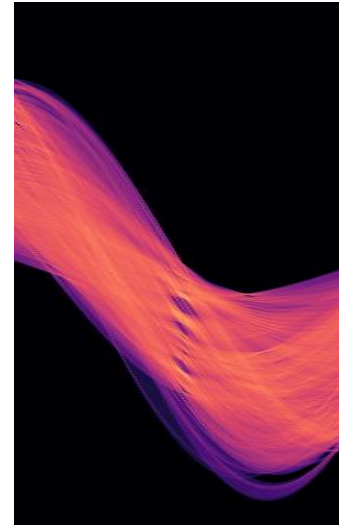
Real-world example



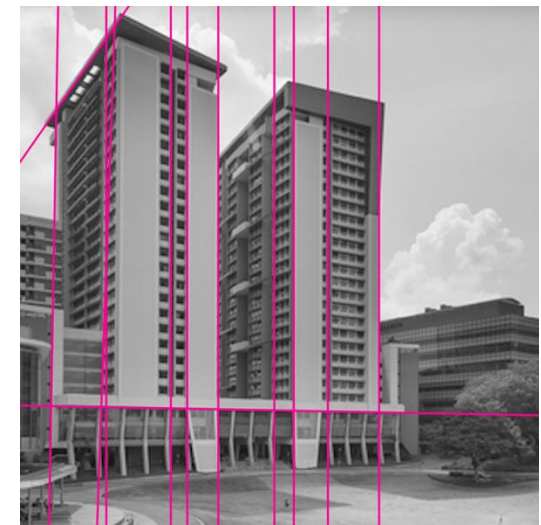
Original



Edges



Hough Space



Hough Lines

Summary

- **Image gradients measure local intensity change.**
Gx and Gy determine the gradient magnitude and direction; the gradient is normal to the corresponding edge.
- **Differentiation amplifies noise.**
Gaussian-derivative filters combine smoothing and differentiation, with sigma controlling the trade-off between noise suppression and edge localization.
- **First- and second-derivative filters describe edges differently.**
Gradient magnitude produces a peak at an edge, while the Laplacian produces a zero-crossing.
- **Canny converts gradient responses into thin, connected edges.**
Combines Gaussian smoothing, gradient estimation, non-maximum suppression, hysteresis threshold and edge linking.
- **The Hough transform groups local edge evidence into global lines.**
Edge pixels vote in a line-parameter space, where peaks represent lines supported by multiple pixels.

Extra Readings

- [Szeliski](#) 7.2.1 Edge Detection
7.4.2 Hough Transforms
- [Torralba](#) Chpt. 18 Image Derivatives (18.2, 18.5, 18.8-9, 18.11)
Chpt. 17 Blur Filters (17.3)
- [Klette](#) 2.3.3, 2.4.1 Edge detector, Gaussian derivatives & Laplacian
3.4 Hough transforms for lines (optionally circles)
- [OpenCV](#) Documentation on Hough Line Transform

AI-Generated FAQs

GRADIENTS AND SOBEL FILTERS

Q: Why do derivative kernels sum to zero? Do they need to be normalized?

A: A derivative should produce zero in a constant-intensity region. A zero-sum kernel guarantees this: adding the same constant to every pixel does not change the response.

Scaling a derivative kernel changes the numerical magnitude of the response, but not where the edges occur. It may, however, require corresponding changes to any thresholds. By contrast, smoothing kernels normally sum to one so that they preserve constant intensities.

Q: Why does the Sobel x-filter respond strongly to vertical edges?

A: G_x measures how intensity changes as we move in the x-direction. Crossing a vertical edge requires moving horizontally, so a vertical edge produces a strong G_x response. The direction in which the filter differentiates and the orientation of the detected edge are perpendicular.

Q: Why does a Sobel filter have negative and positive weights?

The filter compares the weighted average intensity on one side of a pixel with the weighted average on the other. The positive and negative weights implement this subtraction. A step edge therefore produces a strong response. A thin bright bar contains two edges and generally produces two responses with opposite signs.

Q: Why does Sobel use the weights 1, 2, 1 instead of 1, 1, 1?

The Sobel operator combines differentiation in one direction with smoothing in the perpendicular direction. The weights 1, 2, 1 provide this smoothing by placing greater weight on the central row or column.

Q: What exactly does gradient orientation describe?

A: The gradient points in the direction of the greatest increase in intensity, normally from dark to bright. It is perpendicular to the local edge. The edge orientation is therefore the gradient orientation plus or minus 90 degrees. In computation, use: $\theta = \text{atan2}(G_y, G_x)$. This preserves the correct quadrant and also handles $G_x = 0$.

Q: What happens to negative filter responses when the result is displayed?

A: Negative responses should be retained during computation. They encode the polarity of an edge and are needed when computing orientation, Laplacians, or sharpening. For display only, the values may be rescaled, shifted so that zero appears as mid-grey, or shown using their absolute magnitude. Clipping negative values to zero before further processing discards information.

GAUSSIAN DERIVATIVES AND LAPLACIANS

Q: Why smooth an image before differentiating it?

A: Differentiation responds strongly to rapid intensity changes, including noise. Gaussian smoothing suppresses small, high-frequency variations before the derivative is computed.

The two operations can be combined: $\text{differentiate}(G * I) = \text{derivative}(G) * I$

This gives a Gaussian-derivative filter with a well-defined scale. A larger Gaussian sigma suppresses more noise but also reduces edge localization and may merge nearby structures.

Q: Are a derivative of Gaussian and a Difference of Gaussians the same thing?

A: No. A Gaussian derivative is obtained by differentiating a Gaussian kernel and is used to estimate image derivatives. A Difference of Gaussians subtracts two Gaussian-smoothed images at different scales. It is commonly used as an approximation to a Laplacian-of-Gaussian response. For this reason, “DoG” should normally be reserved for Difference of Gaussians.

Q: Why do first- and second-derivative filters mark an edge differently?

A: At an ideal step edge, the first derivative reaches a maximum or minimum. A second derivative changes sign at the edge, so the edge is associated with a zero-crossing.

Consequently, gradient-based detectors look for strong first-derivative magnitude, whereas Laplacian-based detectors often look for zero-crossings after smoothing.

Why is the Laplacian sometimes added to an image and sometimes subtracted for sharpening?

It depends on the sign convention used for the Laplacian kernel. With one convention, a bright local maximum gives a negative Laplacian response, so sharpening subtracts the Laplacian. Reversing every kernel sign reverses this rule.

The important idea is to reinforce local differences. Signed Laplacian values must be retained until they are combined with the original image.

AI-Generated FAQs

CANNY EDGE DETECTION

Q: Why does non-maximum suppression compare pixels along the gradient direction?

A: The gradient direction crosses the edge and therefore crosses the ridge in the gradient magnitude image. Comparing values in this direction retains the peak of the ridge and suppresses neighbouring responses. Comparing along the edge direction would move along the ridge and would not reliably thin it.

Q: Is non-maximum suppression applied separately to the low and high thresholds?

A: No. Non-maximum suppression is applied once to the gradient-magnitude image. The high and low thresholds are then applied to this thinned response. The result is divided into strong edges, weak candidate edges, and rejected pixels.

Q: What exactly happens during hysteresis linking?

Strong edge pixels act as seeds. A weak pixel is retained if it is connected, usually through an 8-connected chain of weak pixels, to a strong edge. The connection may be indirect. A weak pixel does not need to touch a strong pixel immediately, provided that a path of accepted weak pixels eventually reaches one.

HOUGH TRANSFORM

Q: Why use the normal form of a line instead of $y = mx + b$?

A: The slope-intercept form cannot represent vertical lines without an infinite slope, and its parameter range is inconvenient. The normal form, $\rho = x \cos(\theta) + y \sin(\theta)$, represents vertical, horizontal, and diagonal lines using finite parameters with bounded θ .

Q: Why does one image point become a sinusoid in Hough space?

Q: Infinitely many lines can pass through one image point. For each possible θ , the equation gives the corresponding ρ . Plotting these possible parameter pairs produces a sinusoidal curve. The curve represents a family of candidate lines, not a curve present in the original image.

Q: Why does an image line become a peak in the accumulator?

A: Points lying on the same image line generate Hough curves that pass through the same parameter pair. Their votes accumulate near that location, producing a peak. Thus:

image point $\rightarrow$ curve in parameter space

image line $\rightarrow$ peak in parameter space

Q: Why is the Hough result not exact?

A: The accumulator discretizes ρ and θ into bins. Measurement noise, imperfect edge localization, thick edges, and finite bin widths spread votes across neighbouring cells.

Fine bins give more precise parameters but make peaks more fragmented and sensitive to noise. Coarse bins produce more stable but less precise detections.

Q: Can gradient orientation make Hough voting more efficient?

A: Yes. The gradient at an edge is approximately normal to the edge. Instead of voting over every possible θ , an edge pixel can vote only near its measured gradient orientation. This reduces computation and unrelated votes, although orientation noise means that a small angular tolerance is normally required.